

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

NHẬP MÔN HỌC MÁY
ĐỒ ÁN 2

PHÂN LOẠI ĐA LỚP VÀ CÁC TIỀN BỘ MỚI

STT	Họ và tên	MSSV	Vai trò
1	Nguyễn Kim Quốc	23120347	Nhóm trưởng
2	Ngô Thị Thực Quyên	23120348	Thành viên
3	Cao Quốc Tuấn	23120390	Thành viên
4	Lục Hoàng Tuấn	23120393	Thành viên
5	Huỳnh Trọng Viên	23120403	Thành viên

Môn học: Nhập môn Học máy (CSC14005)

Học kỳ: Học kỳ 2 – 2025/2026

GVHD: Th.S Lê Nhật Nam

Bộ môn: Khoa học Máy tính

Mục lục

Tóm tắt	1
Đóng góp chính	1
1 Tổng quan bài toán phân loại đa lớp	2
2 Bảng tổng hợp công thức và ký hiệu	2
2.1 Bảng tổng hợp ký hiệu	3
2.2 Bảng tổng hợp công thức	5
3 Cơ sở lý thuyết	7
3.1 Phát biểu bài toán	7
3.2 Hàm điểm số và quy tắc quyết định	7
3.3 Lề đa lớp	8
3.4 Hàm mất mát lề kinh nghiệm	9
3.5 Độ phức tạp Rademacher và hàm max	10
3.6 Giới hạn tổng quát hóa	12
4 Các thuật toán không kết hợp	16
4.1 Multi-class SVM	16
4.2 Boosting đa lớp – AdaBoost.MH	19
4.3 Cây quyết định (Decision Trees)	20
5 Nhóm thuật toán kết hợp	23
5.1 Động lực	23
5.2 One-vs-All	24
5.3 One-vs-One	25
5.3.1 Biểu quyết có trọng số	28
5.4 Error-Correcting Output Codes	28
5.5 Dự đoán có cấu trúc (Structured Prediction)	29
6 Ứng dụng Deep Learning cho bài toán Multi-class / Multi-label	31
6.1 Đầu ra và hàm mất mát trong mạng nơ-ron	32
6.2 Các kiến trúc học sâu tiêu biểu	34
6.3 Liên hệ với các phương pháp truyền thống	35
6.4 Nhận xét tổng quan	36
7 Các nghiên cứu tiên tiến trong phân loại đa lớp cực đại	36
7.1 Tổng quan Extreme Multi-label Classification	36
7.2 Phân tích các bài báo tiêu biểu	37

7.2.1	Bonsai: Diverse and shallow trees for XMC	37
7.2.2	LightXML: Transformer with Dynamic Negative Sampling	39
7.2.3	XR-Transformer: Fast Multi-Resolution Fine-tuning	40
7.2.4	ICXML: In-Context Learning for Zero-shot XMC	40
7.3	Đánh giá xu hướng phát triển	41
7.3.1	Từ đặc trưng thừa sang ngữ nghĩa sâu	41
7.3.2	Cấu trúc cây vẫn là cốt lõi của tính toán	42
7.3.3	Tập trung vào phân phối đuôi dài	42
7.4	Hạn chế của các phương pháp hiện tại và đề xuất cải tiến	43
7.4.1	Hạn chế hiện tại	43
7.4.2	Đề xuất cải tiến	43
8	Cải tiến của thuật toán cấu trúc: Kết hợp Mạng nơ-ron với CRF (Neural CRFs)	44
8.1	Động lực và bối cảnh	45
8.2	Kiến trúc BiLSTM-CRF – Nền tảng của Neural CRF	45
8.3	Các bài báo tiêu biểu sau năm 2021	46
8.4	Nhận xét tổng quan về xu hướng phát triển	49
9	Thiết kế demo và đánh giá	50
9.1	Mục tiêu thực nghiệm	50
9.2	Mô tả tập dữ liệu	50
9.2.1	Dữ liệu tổng hợp (Synthetic Data)	50
9.3	Dataset đơn nhãn – Fashion-MNIST	51
9.3.1	Dataset đa nhãn – EUR-Lex	51
9.3.2	Quy trình tiền xử lý	52
9.4	Mô hình baseline không kết hợp	52
9.5	Demo thuật toán kết hợp	53
9.6	Thước đo đánh giá	53
9.7	Bảng so sánh đánh giá thực nghiệm	54
9.8	Thực nghiệm minh họa nhóm thuật toán không kết hợp	55
9.8.1	Cây quyết định (Decision Tree)	55
9.8.2	Multi-class SVM (Crammer-Singer Primal)	56
9.8.3	AdaBoost.MH và Thủ thuật Cách ly Đặc trưng	58
9.9	Thực nghiệm minh họa thuật toán kết hợp	59
9.9.1	One-vs-All (OVA) và Hiệu chuẩn Platt Scaling	59
9.9.2	One-vs-One (OVO) và Phân tích Nhầm lẫn Cặp	60
9.9.3	ECOC và Sức mạnh Sửa lỗi	62
9.9.4	So sánh độ phức tạp thời gian và kiểm định thống kê	63
9.10	Phân tích kết quả kỳ vọng	63

10 Kết luận

64

Tóm tắt

Báo cáo trình bày có hệ thống bài toán phân loại đa lớp, từ phát biểu hình thức, các hàm mất mát và giới hạn tổng quát hóa đến những nhóm thuật toán tiêu biểu. Nội dung tập trung vào hai hướng chính: các mô hình học trực tiếp trên không gian nhãn đa lớp và các phương pháp quy đổi thành nhiều bài toán nhị phân như One-vs-All, One-vs-One và Error-Correcting Output Codes. Bên cạnh nền tảng lý thuyết, báo cáo phân tích các tiến bộ gần đây trong phân loại đa nhãn cực lớn, bao gồm mô hình dựa trên cây nhãn, biểu diễn nhãn, Transformer và chiến lược tạo danh sách ứng viên. Cuối cùng, báo cáo đề xuất một thiết kế demo thực nghiệm nhằm so sánh các phương pháp theo độ chính xác, macro-F1, chi phí huấn luyện và khả năng mở rộng.

Từ khóa: phân loại đa lớp, phân loại đa nhãn cực lớn, One-vs-All, One-vs-One, ECOC, cây nhãn, Transformer.

Đóng góp chính

- Hệ thống hóa cơ sở lý thuyết của phân loại đa lớp, bao gồm ký hiệu, hàm mất mát, đánh giá và các giới hạn tổng quát hóa.
- So sánh các nhóm thuật toán trực tiếp và kết hợp theo số mô hình, chi phí tối ưu, cơ chế dự đoán và hạn chế thực nghiệm.
- Phân tích các hướng nghiên cứu hiện đại cho bài toán nhãn cực lớn, đặc biệt là cây nhãn, nhúng nhãn, mô hình Transformer và cơ chế shortlist–rerank.
- Đề xuất quy trình demo thực nghiệm để đánh giá đồng thời chất lượng dự đoán, tính ổn định và khả năng mở rộng của các phương pháp.

1 Tổng quan bài toán phân loại đa lớp

Phân loại đa lớp (multi-class classification) là bài toán học có giám sát trong đó mỗi điểm dữ liệu $\mathbf{x} \in \mathcal{X}$ được gán đúng một nhãn thuộc tập $\mathcal{Y} = \{1, 2, \dots, k\}$, với $k > 2$. Khác với phân loại nhị phân, mô hình phải so sánh đồng thời nhiều lớp và đưa ra quyết định cuối cùng dựa trên điểm số, xác suất hoặc cơ chế biểu quyết.

Cần phân biệt hai thiết lập thường gặp:

- **Đơn nhãn (mono-label):** mỗi mẫu thuộc đúng một lớp, ví dụ phân loại ảnh chữ số từ 0 đến 9.
- **Đa nhãn (multi-label):** mỗi mẫu có thể đồng thời thuộc nhiều lớp, ví dụ một văn bản vừa thuộc nhãn “thể thao” vừa thuộc nhãn “sức khỏe”.

Các thách thức chính của phân loại đa lớp gồm chi phí tính toán tăng theo số lớp, mất cân bằng lớp, độ phức tạp của biên quyết định và khả năng tồn tại quan hệ phân cấp hoặc quan hệ đồ thị giữa các nhãn. Khi k lớn, chiến lược học trực tiếp trên toàn bộ không gian nhãn thường gặp trở ngại về tối ưu và bộ nhớ, trong khi các chiến lược quy về nhiều bài toán nhị phân cần cơ chế tổng hợp dự đoán hiệu quả.

Lưu ý

Trong đồ án này, trọng tâm là cơ sở lý thuyết của phân loại đa lớp, các nhóm thuật toán không kết hợp và kết hợp, các tiến bộ gần đây cho bài toán nhãn cực lớn, cùng một thiết kế demo thực nghiệm để so sánh các hướng tiếp cận.

2 Bảng tổng hợp công thức và ký hiệu

Bảng dưới đây tóm tắt các ký hiệu và công thức quan trọng được sử dụng xuyên suốt báo cáo. Mục tiêu của phần này là giúp người đọc tra cứu nhanh các biểu thức chính trước khi đi vào phần chứng minh và phân tích thuật toán.

2.1 Bảng tổng hợp ký hiệu

Bảng 1: Tổng hợp các ký hiệu chính trong báo cáo

Nhóm	Ký hiệu	Ý nghĩa
Dữ liệu	m	Số lượng điểm dữ liệu huấn luyện.
Dữ liệu	N	Số chiều đặc trưng của dữ liệu đầu vào.
Dữ liệu	$\mathbf{x}_i$	Điểm dữ liệu thứ i .
Dữ liệu	y_i	Nhãn đúng của điểm dữ liệu thứ i trong bài toán đơn nhãn.
Dữ liệu	$\mathcal{X}$	Không gian đầu vào.
Dữ liệu	$\mathcal{Y}$	Tập nhãn/lớp của bài toán.
Dữ liệu	$k = \mathcal{Y} $	Số lượng lớp trong bài toán phân loại đa lớp.
Dữ liệu	L	Số nhãn trong bài toán extreme multi-label classification.
Hàm điểm và lề	$h(\mathbf{x}, y)$	Hàm điểm số của mẫu $\mathbf{x}$ đối với nhãn y .
Hàm điểm và lề	$g_h(\mathbf{x})$	Hàm dự đoán nhãn cuối cùng sinh bởi hàm điểm h .

Bảng 2: Tổng hợp các ký hiệu chính trong báo cáo (tiếp)

Nhóm	Ký hiệu	Ý nghĩa
Hàm điểm và lề	$\rho_h(\mathbf{x}, y)$	Lề đa lớp tại mẫu $(\mathbf{x}, y)$.
Hàm điểm và lề	ρ	Ngưỡng lề mong muốn trong hàm mất mát lề.
Hàm điểm và lề	Φ_ρ	Hàm mất mát lề với tham số lề ρ .
Hàm điểm và lề	$R(h)$	Sai số thật/kỳ vọng của giả thuyết h .
Hàm điểm và lề	$\widehat{R}_\rho(h)$	Sai số lề kinh nghiệm trên tập huấn luyện.
Rademacher	S	Tập mẫu dùng để tính độ phức tạp Rademacher kinh nghiệm.
Rademacher	σ_i	Biến Rademacher độc lập, nhận giá trị $\{-1, +1\}$.
Rademacher	$\widehat{\mathfrak{R}}_S(\mathcal{F})$	Độ phức tạp Rademacher kinh nghiệm của lớp hàm $\mathcal{F}$.
Rademacher	$\mathfrak{R}_m(\mathcal{F})$	Độ phức tạp Rademacher kỳ vọng theo mẫu kích thước m .
Rademacher	$\Pi_1(\mathcal{H})$	Lớp hàm tọa độ thu được khi cố định một nhãn của $h \in \mathcal{H}$.

Bảng 3: Tổng hợp các ký hiệu chính trong báo cáo (tiếp)

Nhóm	Ký hiệu	Ý nghĩa
Kernel/SVM	$\Phi(\mathbf{x}_i)$	Ánh xạ đặc trưng của $\mathbf{x}_i$ vào không gian Hilbert.
Kernel/SVM	$K(\mathbf{x}_i, \mathbf{x}_j)$	Hàm kernel, bằng tích vô hướng của hai ánh xạ đặc trưng.
Kernel/SVM	$\mathbf{w}_l$	Vector trọng số tương ứng với lớp l .
Kernel/SVM	W	Ma trận trọng số ghép từ $(\mathbf{w}_1, \dots, \mathbf{w}_k)$.
Kernel/SVM	ξ_i	Biến lỗi lỏng biểu diễn mức vi phạm lề tại mẫu $\mathbf{x}_i$.
Kernel/SVM	C	Siêu tham số điều chuẩn trong Multi-class SVM.
Kernel/SVM	Λ	Chặn trên chuẩn của ma trận trọng số W .
Kernel/SVM	r	Chặn trên chuẩn đặc trưng, với $K(\mathbf{x}, \mathbf{x}) \leq r^2$.
Boosting	h_t	Bộ phân loại cơ sở/yếu tại vòng lặp t .
Boosting	α_t	Trọng số của bộ phân loại yếu h_t .

Bảng 4: Tổng hợp các ký hiệu chính trong báo cáo (tiếp)

Nhóm	Ký hiệu	Ý nghĩa
Boosting	g_n	Tổ hợp các bộ phân loại yếu sau n vòng.
Boosting	$F(\boldsymbol{\alpha})$	Hàm mục tiêu mũ cần tối ưu trong AdaBoost.MH.
Boosting	$D_t(i, l)$	Trọng số của cặp mẫu–nhân (i, l) tại vòng t .
Cây quyết định	n	Một nút trong cây quyết định.
Cây quyết định	q	Câu hỏi phân tách tại nút, ví dụ $X_j \leq a$.
Cây quyết định	$p_l(n)$	Tỷ lệ mẫu lớp l rơi vào nút n .
Cây quyết định	$F(n)$	Hàm đo độ vẩn đục tại nút n .
Cây quyết định	$\tilde{F}(n, q)$	Mức giảm độ vẩn đục khi tách nút n bằng câu hỏi q .
Cây quyết định	λ	Tham số điều chuẩn dùng để cắt tỉa cây.
ECOC/XMC	M	Ma trận mã ECOC, mỗi hàng là codeword của một lớp.

Bảng 5: Tổng hợp các ký hiệu chính trong báo cáo (tiếp)

Nhóm	Ký hiệu	Ý nghĩa
ECOC/XMC	M_l	Codeword ứng với lớp l .
ECOC/XMC	d_H	Khoảng cách Hamming giữa hai vector mã.
ECOC/XMC	c	Độ dài mã ECOC, tương ứng số bộ phân loại nhị phân.

2.2 Bảng tổng hợp công thức

Bảng 6: Tổng hợp các công thức chính trong báo cáo

Chủ đề	Công thức	Ý nghĩa
Hàm điểm số	$h : \mathcal{X} \times \mathcal{Y} \rightarrow \mathbb{R}$	Gán điểm tin cậy cho cặp dữ liệu–nhãn $(\mathbf{x}, y)$.
Quy tắc dự đoán đa lớp	$g_h(\mathbf{x}) = \arg \max_{y \in \mathcal{Y}} h(\mathbf{x}, y)$	Chọn nhãn có điểm số cao nhất.
Lề đa lớp	$\rho_h(\mathbf{x}, y) = h(\mathbf{x}, y) - \max_{y' \neq y} h(\mathbf{x}, y')$	Khoảng cách giữa điểm nhãn đúng và nhãn sai cạnh tranh mạnh nhất.
Điều kiện phân loại sai	$g_h(\mathbf{x}) \neq y \iff \rho_h(\mathbf{x}, y) \leq 0$	Mô hình sai khi lề không dương.
Sai số 0–1 kinh nghiệm	$\widehat{R}_{0/1}(h) = \frac{1}{m} \sum_{i=1}^m \mathbb{1}[\rho_h(\mathbf{x}_i, y_i) \leq 0]$	Tỷ lệ mẫu huấn luyện bị phân loại sai.
Hàm mất mát lề	$\Phi_\rho(t) = \begin{cases} 1, & t \leq 0 \\ 1 - t/\rho, & 0 < t < \rho \\ 0, & t \geq \rho \end{cases}$	Phạt mẫu sai hoặc đúng nhưng chưa đạt lề mong muốn.

Bảng 7: Tổng hợp các công thức chính trong báo cáo (tiếp)

Chủ đề	Công thức	Ý nghĩa
Sai số lẻ kinh nghiệm	$\widehat{R}_\rho(h) = \frac{1}{m} \sum_{i=1}^m \Phi_\rho(\rho_h(\mathbf{x}_i, y_i))$	Chặn trên mềm của sai số huấn luyện 0–1.
Độ phức tạp Rademacher	$\widehat{\mathfrak{R}}_S(\mathcal{F}) = \frac{1}{m} \mathbb{E}_\sigma [\sup_{f \in \mathcal{F}} \sum_{i=1}^m \sigma_i f(\mathbf{x}_i)]$	Đo khả năng lớp hàm khớp nhiều ngẫu nhiên.
Bổ đề max	$\widehat{\mathfrak{R}}_S(\mathcal{G}) \leq \sum_{j=1}^\ell \widehat{\mathfrak{R}}_S(\mathcal{F}_j)$	Chặn độ phức tạp của lớp hàm lấy cực đại.
Giới hạn tổng quát hóa đa lớp	$R(h) \leq \widehat{R}_\rho(h) + \frac{2k^2}{\rho} \mathfrak{R}_m(\Pi_1(\mathcal{H})) + \sqrt{\frac{\log(1/\delta)}{2m}}$	Chặn sai số thật bằng sai số lẻ, độ phức tạp mô hình và xác suất tin cậy.
Kernel bound	$\mathfrak{R}_m(\Pi_1(\mathcal{H}_{K,p})) \leq \sqrt{\frac{r^2 \Lambda^2}{m}}$	Chặn Rademacher cho giả thuyết kernel khi $\ W\ \leq \Lambda$ và $K(\mathbf{x}, \mathbf{x}) \leq r^2$.
Hệ quả kernel	$R(h) \leq \widehat{R}_\rho(h) + 2k^2 \sqrt{\frac{r^2 \Lambda^2}{m \rho^2}} + \sqrt{\frac{\log(1/\delta)}{2m}}$	Dạng cụ thể của giới hạn lẻ cho mô hình kernel.

Bảng 8: Tổng hợp các công thức chính trong báo cáo (tiếp)

Chủ đề	Công thức	Ý nghĩa
Multi-class SVM primal	$\min_{W, \xi} \frac{1}{2} \sum_{l=1}^k \ \mathbf{w}_l\ ^2 + C \sum_{i=1}^m \xi_i$	Tối ưu đồng thời lẻ lớn và lỗi huấn luyện nhỏ.
Ràng buộc SVM	$\langle \mathbf{w}_{y_i}, \Phi(\mathbf{x}_i) \rangle \geq \langle \mathbf{w}_l, \Phi(\mathbf{x}_i) \rangle + 1 - \xi_i$	Điểm nhãn đúng phải lớn hơn mọi nhãn sai ít nhất $1 - \xi_i$.
OVA	$h_{\text{OVA}}(\mathbf{x}) = \arg \max_{l \in \mathcal{Y}} f_l(\mathbf{x})$	Huấn luyện một bộ phân loại cho mỗi lớp so với phần còn lại.
Số mô hình OVO	$\binom{k}{2} = \frac{k(k-1)}{2}$	Số bộ phân loại cặp lớp cần huấn luyện.
ECOC decoding	$h_{\text{ECOC}}(\mathbf{x}) = \arg \min_{l \in \mathcal{Y}} d_H(M_l, \mathbf{h}(\mathbf{x}))$	Chọn lớp có mã gần nhất với vector dự đoán.
Khoảng cách Hamming	$d_H(\mathbf{u}, \mathbf{v}) = \frac{1}{2} \sum_{j=1}^c (1 - u_j v_j)$	Số bit khác nhau giữa hai codeword nhị phân.

Bảng 9: Tổng hợp các công thức chính trong báo cáo (tiếp)

Chủ đề	Công thức	Ý nghĩa
AdaBoost.MH	$F(\boldsymbol{\alpha}) = \sum_{i=1}^m \sum_{l=1}^k \exp(-y_i[l]g_n(\mathbf{x}_i, l))$	Hàm mục tiêu mũ cho bài toán đa nhãn.
Cập nhật trọng số Boosting	$D_{t+1}(i, l) \propto D_t(i, l) \exp(-\alpha_t y_i[l]h_t(\mathbf{x}_i, l))$	Tăng trọng số cho các cặp mẫu–nhãn dự đoán sai.
Entropy	$\text{Entropy}(n) = -\sum_{l=1}^k p_l(n) \log p_l(n)$	Độ vắn đục của nút trong cây quyết định.
Gini index	$\text{Gini}(n) = 1 - \sum_{l=1}^k p_l(n)^2$	Độ vắn đục phổ biến khi xây cây quyết định.
Cắt tỉa cây	$G_\lambda(\text{tree}) = \sum_{n \in \widetilde{\text{tree}}} n F(n) + \lambda \widetilde{\text{tree}} $	Cân bằng giữa lỗi tại lá và độ phức tạp của cây.

3 Cơ sở lý thuyết

3.1 Phát biểu bài toán

Xét không gian đầu vào $\mathcal{X}$ và tập nhãn hữu hạn $\mathcal{Y} = \{1, 2, \dots, k\}$ với $k > 2$. Tập huấn luyện gồm m mẫu độc lập cùng phân phối

$$S = \{(\mathbf{x}_i, y_i)\}_{i=1}^m, \quad \mathbf{x}_i \in \mathcal{X}, y_i \in \mathcal{Y}.$$

Mục tiêu là học một hàm phân loại $g : \mathcal{X} \rightarrow \mathcal{Y}$ sao cho sai số kỳ vọng

$$R(g) = \mathbb{P}_{(\mathbf{x}, y) \sim \mathcal{D}}[g(\mathbf{x}) \neq y]$$

nhỏ trên dữ liệu chưa thấy. Khác với phân loại nhị phân, mô hình đa lớp phải đồng thời so sánh nhiều nhãn, do đó cách biểu diễn đầu ra và cách thiết kế hàm mất mát đóng vai trò rất quan trọng.

Trong thiết lập **đơn nhãn**, mỗi mẫu thuộc đúng một lớp. Trong thiết lập **đa nhãn**, mỗi mẫu có thể ứng với một tập nhãn $Y \subseteq \mathcal{Y}$. Phần lý thuyết dưới đây tập trung vào đơn nhãn, sau đó các mở rộng đa nhãn và nhãn cực lớn được trình bày ở các phần sau.

3.2 Hàm điểm số và quy tắc quyết định

Một cách mô hình hóa phổ biến là sử dụng hàm điểm số

$$h : \mathcal{X} \times \mathcal{Y} \rightarrow \mathbb{R},$$

trong đó $h(\mathbf{x}, y)$ biểu thị mức độ tin cậy rằng mẫu $\mathbf{x}$ thuộc lớp y . Quy tắc dự đoán là

$$\hat{y} = g_h(\mathbf{x}) = \arg \max_{y \in \mathcal{Y}} h(\mathbf{x}, y).$$

Nếu tồn tại nhiều nhãn đạt cùng điểm số lớn nhất, mô hình cần một quy tắc phá hòa, ví dụ chọn nhãn có xác suất hậu nghiệm lớn hơn, chọn nhãn xuất hiện nhiều hơn trên validation set, hoặc chọn nhãn có chỉ số nhỏ nhất để bảo đảm tính xác định.

Với mô hình tuyến tính, một dạng thường gặp là

$$h(\mathbf{x}, y) = \mathbf{w}_y^\top \phi(\mathbf{x}),$$

trong đó $\phi(\mathbf{x})$ là ánh xạ đặc trưng và $\mathbf{w}_y$ là vector trọng số của lớp y . Khi dùng kernel, tích vô hướng $\phi(\mathbf{x})^\top \phi(\mathbf{x}')$ được thay bằng hàm kernel $K(\mathbf{x}, \mathbf{x}')$.

3.3 Lề đa lớp

Trong phân loại đa lớp, một giả thuyết không nhất thiết dự đoán nhãn trực tiếp. Thay vào đó, giả thuyết được biểu diễn thông qua hàm tính điểm

$$h : \mathcal{X} \times \mathcal{Y} \rightarrow \mathbb{R},$$

trong đó $h(\mathbf{x}, y)$ là điểm số mà mô hình gán cho nhãn y tại điểm dữ liệu $\mathbf{x}$. Nhãn dự đoán được chọn theo quy tắc điểm số lớn nhất:

$$g_h(\mathbf{x}) = \arg \max_{y \in \mathcal{Y}} h(\mathbf{x}, y).$$

Với mẫu đã gán nhãn $(\mathbf{x}, y)$, lề đa lớp của h tại $(\mathbf{x}, y)$ được định nghĩa là

$$\rho_h(\mathbf{x}, y) = h(\mathbf{x}, y) - \max_{y' \neq y} h(\mathbf{x}, y').$$

Thành phần $\max_{y' \neq y} h(\mathbf{x}, y')$ là điểm số của nhãn sai nguy hiểm nhất, tức nhãn sai cạnh tranh mạnh nhất với nhãn đúng. Vì vậy, lề đo khoảng cách giữa điểm của nhãn đúng và điểm của đối thủ mạnh nhất.

Từ định nghĩa trên, ta có nhận xét quan trọng:

$$g_h(\mathbf{x}) \neq y \iff \rho_h(\mathbf{x}, y) \leq 0,$$

nếu quy tắc phá hòa được xem là bất lợi cho nhãn đúng. Khi $\rho_h(\mathbf{x}, y) > 0$, mô hình phân loại đúng; khi $\rho_h(\mathbf{x}, y)$ càng lớn, mô hình càng tự tin vì nhãn đúng tách xa mọi nhãn sai.

Lưu ý

Lề đa lớp khác với lề nhị phân ở chỗ nó không chỉ so sánh với một ranh giới quyết định duy nhất, mà so sánh nhãn đúng với toàn bộ $k - 1$ nhãn còn lại. Đây

là lý do các giới hạn lý thuyết cho phân loại đa lớp thường phụ thuộc vào số lớp k .

Ví dụ, giả sử một mẫu có nhãn đúng là lớp 2 và mô hình cho điểm số

$$(h(\mathbf{x}, 1), h(\mathbf{x}, 2), h(\mathbf{x}, 3), h(\mathbf{x}, 4)) = (0.4, 1.2, 0.9, 0.1).$$

Khi đó

$$\rho_h(\mathbf{x}, 2) = 1.2 - \max\{0.4, 0.9, 0.1\} = 1.2 - 0.9 = 0.3.$$

Mẫu được phân loại đúng nhưng lề chỉ bằng 0.3, nghĩa là lớp 3 vẫn là đối thủ khá gần. Nếu điểm của lớp 3 tăng nhẹ vượt quá 1.2, mô hình sẽ phân loại sai.

3.4 Hàm mất mát lề kinh nghiệm

Sai số huấn luyện 0–1 chỉ quan tâm mô hình đúng hay sai:

$$\widehat{R}_{0/1}(h) = \frac{1}{m} \sum_{i=1}^m \mathbb{I}[\rho_h(\mathbf{x}_i, y_i) \leq 0].$$

Tuy nhiên, đại lượng này không phân biệt mẫu đúng với lề lớn và mẫu đúng sát biên quyết định. Để phân tích khả năng tổng quát hóa, ta dùng hàm mất mát lề với một mức lề mong muốn $\rho > 0$.

Hàm mất mát lề Φ_ρ được định nghĩa bởi

$$\Phi_\rho(t) = \begin{cases} 1, & t \leq 0, \\ 1 - \frac{t}{\rho}, & 0 < t < \rho, \\ 0, & t \geq \rho. \end{cases}$$

Hàm này có ba vùng ý nghĩa:

- nếu $t \leq 0$, mẫu bị phân loại sai hoặc hòa với nhãn sai nên bị phạt tối đa;
- nếu $0 < t < \rho$, mẫu đúng nhưng chưa đủ tự tin nên bị phạt một phần;
- nếu $t \geq \rho$, mẫu đúng với lề đủ lớn nên không bị phạt.

Sai số lề kinh nghiệm của h trên tập huấn luyện $S = \{(\mathbf{x}_i, y_i)\}_{i=1}^m$ là

$$\widehat{R}_\rho(h) = \frac{1}{m} \sum_{i=1}^m \Phi_\rho(\rho_h(\mathbf{x}_i, y_i)).$$

Do $\Phi_\rho(t) \geq \mathbb{I}[t \leq 0]$, ta có

$$\widehat{R}_{0/1}(h) \leq \widehat{R}_\rho(h).$$

Nói cách khác, hàm mất mát lẻ là một chặn trên của sai số huấn luyện. Nó phạt cả các điểm bị phân loại sai và các điểm phân loại đúng nhưng có confidence nhỏ hơn hoặc bằng ngưỡng lẻ mong muốn.

Một tính chất quan trọng là Φ_ρ có hằng số Lipschitz bằng $1/\rho$:

$$|\Phi_\rho(a) - \Phi_\rho(b)| \leq \frac{1}{\rho}|a - b|.$$

Tính chất này cho phép chuyển bài toán chặn sai số thật sang chặn độ phức tạp Rademacher của lớp hàm lẻ. Khi ρ lớn, hệ số Lipschitz nhỏ hơn, nhưng điều kiện để mẫu có mất mát bằng 0 cũng khắt khe hơn. Vì vậy tồn tại đánh đổi giữa việc chọn lẻ lớn và sai số lẻ kinh nghiệm.

3.5 Độ phức tạp Rademacher và hàm max

Độ phức tạp Rademacher đo khả năng một lớp hàm khớp với nhiều ngẫu nhiên. Với lớp hàm $\mathcal{F}$ và mẫu $S = (\mathbf{x}_1, \dots, \mathbf{x}_m)$, độ phức tạp Rademacher kinh nghiệm là

$$\hat{\mathfrak{R}}_S(\mathcal{F}) = \frac{1}{m} \mathbb{E}_\sigma \left[\sup_{f \in \mathcal{F}} \sum_{i=1}^m \sigma_i f(\mathbf{x}_i) \right],$$

trong đó σ_i là các biến Rademacher độc lập, nhận giá trị $+1$ hoặc -1 với xác suất bằng nhau. Nếu $\hat{\mathfrak{R}}_S(\mathcal{F})$ lớn, lớp hàm đủ linh hoạt để khớp nhiều tốt, do đó nguy cơ overfitting cao hơn.

Trong phân tích đa lớp, lẻ chứa toán tử

$$\max_{y' \neq y} h(\mathbf{x}, y'),$$

nên ta cần một bổ đề để chặn độ phức tạp của lớp hàm tạo bởi phép lấy cực đại.

Mệnh đề 3.1 (Bổ đề 8.1: Độ phức tạp Rademacher của hàm max). *Cho ℓ tập giả thuyết $\mathcal{F}_1, \dots, \mathcal{F}_\ell$ và*

$$\mathcal{G} = \{\max\{h_1, \dots, h_\ell\} : h_j \in \mathcal{F}_j, j \in [1, \ell]\}.$$

Với mọi tập mẫu S kích thước m , ta có

$$\hat{\mathfrak{R}}_S(\mathcal{G}) \leq \sum_{j=1}^{\ell} \hat{\mathfrak{R}}_S(\mathcal{F}_j).$$

Chứng minh. Bước 1: Chứng minh cho trường hợp $\ell = 2$. Với mọi $h_1 \in \mathcal{F}_1$ và

$h_2 \in \mathcal{F}_2$, ta dùng hằng đẳng thức

$$\max\{h_1, h_2\} = \frac{1}{2}(h_1 + h_2 + |h_1 - h_2|).$$

Thay vào định nghĩa độ phức tạp Rademacher kinh nghiệm và dùng tính cộng dưới của supremum, ta có

$$\begin{aligned} \widehat{\mathfrak{R}}_S(\mathcal{G}) &= \frac{1}{m} \mathbb{E}_{\boldsymbol{\sigma}} \left[\sup_{h_1, h_2} \sum_{i=1}^m \sigma_i \max\{h_1(\mathbf{x}_i), h_2(\mathbf{x}_i)\} \right] \\ &\leq \frac{1}{2m} \mathbb{E}_{\boldsymbol{\sigma}} \left[\sup_{h_1 \in \mathcal{F}_1} \sum_{i=1}^m \sigma_i h_1(\mathbf{x}_i) \right] + \frac{1}{2m} \mathbb{E}_{\boldsymbol{\sigma}} \left[\sup_{h_2 \in \mathcal{F}_2} \sum_{i=1}^m \sigma_i h_2(\mathbf{x}_i) \right] \\ &\quad + \frac{1}{2m} \mathbb{E}_{\boldsymbol{\sigma}} \left[\sup_{h_1 \in \mathcal{F}_1, h_2 \in \mathcal{F}_2} \sum_{i=1}^m \sigma_i |(h_1 - h_2)(\mathbf{x}_i)| \right] \\ &= \frac{1}{2} \widehat{\mathfrak{R}}_S(\mathcal{F}_1) + \frac{1}{2} \widehat{\mathfrak{R}}_S(\mathcal{F}_2) + T, \end{aligned}$$

trong đó

$$T = \frac{1}{2m} \mathbb{E}_{\boldsymbol{\sigma}} \left[\sup_{h_1, h_2} \sum_{i=1}^m \sigma_i |(h_1 - h_2)(\mathbf{x}_i)| \right].$$

Bước 2: Áp dụng bổ đề co Talagrand. Hàm $u \mapsto |u|$ là hàm 1-Lipschitz và thỏa $|0| = 0$. Theo bổ đề co Talagrand,

$$T \leq \frac{1}{2m} \mathbb{E}_{\boldsymbol{\sigma}} \left[\sup_{h_1, h_2} \sum_{i=1}^m \sigma_i (h_1 - h_2)(\mathbf{x}_i) \right].$$

Tiếp tục dùng tính cộng dưới của supremum:

$$T \leq \frac{1}{2m} \mathbb{E}_{\boldsymbol{\sigma}} \left[\sup_{h_1 \in \mathcal{F}_1} \sum_{i=1}^m \sigma_i h_1(\mathbf{x}_i) \right] + \frac{1}{2m} \mathbb{E}_{\boldsymbol{\sigma}} \left[\sup_{h_2 \in \mathcal{F}_2} \sum_{i=1}^m (-\sigma_i) h_2(\mathbf{x}_i) \right].$$

Vì $(\sigma_1, \dots, \sigma_m)$ và $(-\sigma_1, \dots, -\sigma_m)$ có cùng phân phối, nên

$$T \leq \frac{1}{2} \widehat{\mathfrak{R}}_S(\mathcal{F}_1) + \frac{1}{2} \widehat{\mathfrak{R}}_S(\mathcal{F}_2).$$

Cộng các hạng tử lại, ta thu được

$$\widehat{\mathfrak{R}}_S(\mathcal{G}) \leq \widehat{\mathfrak{R}}_S(\mathcal{F}_1) + \widehat{\mathfrak{R}}_S(\mathcal{F}_2).$$

Bước 3: Mở rộng bằng quy nạp. Với $\ell > 2$, viết

$$\max\{h_1, \dots, h_\ell\} = \max\{h_1, \max(h_2, \dots, h_\ell)\}.$$

Áp dụng kết quả $\ell = 2$ cho $\mathcal{F}_1$ và lớp hàm

$$\mathcal{G}_{2:\ell} = \{\max(h_2, \dots, h_\ell) : h_j \in \mathcal{F}_j, j = 2, \dots, \ell\},$$

rồi dùng giả thiết quy nạp cho $\mathcal{G}_{2:\ell}$, ta có

$$\widehat{\mathfrak{R}}_S(\mathcal{G}) \leq \widehat{\mathfrak{R}}_S(\mathcal{F}_1) + \widehat{\mathfrak{R}}_S(\mathcal{G}_{2:\ell}) \leq \sum_{j=1}^{\ell} \widehat{\mathfrak{R}}_S(\mathcal{F}_j).$$

Bổ đề được chứng minh. □

3.6 Giới hạn tổng quát hóa

Gọi $R(h) = \mathbb{P}[g_h(\mathbf{x}) \neq y]$ là sai số thật và $\widehat{R}_\rho(h)$ là sai số lẻ kinh nghiệm. Đặt $\Pi_1(\mathcal{H})$ là lớp hàm tọa độ thu được khi cố định một nhãn:

$$\Pi_1(\mathcal{H}) = \{\mathbf{x} \mapsto h(\mathbf{x}, y) : h \in \mathcal{H}, y \in \mathcal{Y}\}.$$

Ký hiệu $\mathfrak{R}_m(\cdot)$ là độ phức tạp Rademacher kỳ vọng theo mẫu kích thước m .

Định lý 3.1 (Định lý 8.1: Giới hạn lẻ phân loại đa lớp). *Với mọi $\rho > 0$ và $\delta > 0$, với xác suất ít nhất $1 - \delta$, bất đẳng thức sau đúng đồng thời với mọi $h \in \mathcal{H}$:*

$$R(h) \leq \widehat{R}_\rho(h) + \frac{2k^2}{\rho} \mathfrak{R}_m(\Pi_1(\mathcal{H})) + \sqrt{\frac{\log(1/\delta)}{2m}}.$$

Chứng minh. **Bước 1: Chặn sai số bằng hàm mất mát lẻ.** Gọi $\widetilde{\mathcal{H}}$ là lớp hàm lẻ

$$\widetilde{\mathcal{H}} = \{(\mathbf{x}, y) \mapsto \rho_h(\mathbf{x}, y) : h \in \mathcal{H}\}.$$

Vì hàm chỉ thị sai số luôn bị chặn bởi hàm mất mát lẻ,

$$\mathbb{I}[g_h(\mathbf{x}) \neq y] \leq \Phi_\rho(\rho_h(\mathbf{x}, y)),$$

áp dụng định lý tổng quát hóa dựa trên Rademacher cho lớp $\Phi_\rho \circ \widetilde{\mathcal{H}}$ cho ta, với xác suất ít nhất $1 - \delta$,

$$R(h) \leq \mathbb{E}[\Phi_\rho(\rho_h(\mathbf{x}, y))] \leq \widehat{R}_\rho(h) + 2\mathfrak{R}_m(\Phi_\rho \circ \widetilde{\mathcal{H}}) + \sqrt{\frac{\log(1/\delta)}{2m}}.$$

Do Φ_ρ là hàm $(1/\rho)$ -Lipschitz, bổ đề co Talagrand cho

$$\mathfrak{R}_m(\Phi_\rho \circ \widetilde{\mathcal{H}}) \leq \frac{1}{\rho} \mathfrak{R}_m(\widetilde{\mathcal{H}}).$$

Vì vậy nhiệm vụ còn lại là chặn $\mathfrak{R}_m(\tilde{\mathcal{H}})$.

Bước 2: Phân rã độ phức tạp Rademacher của lớp lẻ. Với mẫu $S = \{(\mathbf{x}_i, y_i)\}_{i=1}^m$, ta có thể viết độ phức tạp của lớp lẻ theo tổng trên các nhãn đúng bằng hàm chỉ thị:

$$\mathfrak{R}_m(\tilde{\mathcal{H}}) \leq \frac{1}{m} \sum_{y \in \mathcal{Y}} \mathbb{E}_{S, \sigma} \left[\sup_{h \in \mathcal{H}} \sum_{i=1}^m \sigma_i \rho_h(\mathbf{x}_i, y) \mathbb{1}_{\{y=y_i\}} \right].$$

Dùng mẹo đổi biến $\mathbb{1}_{\{y=y_i\}} = (\epsilon_i + 1)/2$ với $\epsilon_i \in \{-1, +1\}$ và tính chất σ_i có cùng phân phối với $\sigma_i \epsilon_i$, ta thu được chặn đơn giản hơn:

$$\mathfrak{R}_m(\tilde{\mathcal{H}}) \leq \frac{1}{m} \sum_{y \in \mathcal{Y}} \mathbb{E}_{S, \sigma} \left[\sup_{h \in \mathcal{H}} \sum_{i=1}^m \sigma_i \rho_h(\mathbf{x}_i, y) \right].$$

Bước 3: Khai triển lẻ và áp dụng Bổ đề 8.1. Thay định nghĩa lẻ

$$\rho_h(\mathbf{x}_i, y) = h(\mathbf{x}_i, y) - \max_{y' \neq y} h(\mathbf{x}_i, y')$$

vào biểu thức trên. Với mỗi nhãn y , dùng tính cộng dưới của supremum và việc $-\sigma_i$ có cùng phân phối với σ_i , ta được

$$\begin{aligned} & \frac{1}{m} \mathbb{E}_{S, \sigma} \left[\sup_{h \in \mathcal{H}} \sum_{i=1}^m \sigma_i \rho_h(\mathbf{x}_i, y) \right] \\ & \leq \frac{1}{m} \mathbb{E}_{S, \sigma} \left[\sup_{h \in \mathcal{H}} \sum_{i=1}^m \sigma_i h(\mathbf{x}_i, y) \right] + \frac{1}{m} \mathbb{E}_{S, \sigma} \left[\sup_{h \in \mathcal{H}} \sum_{i=1}^m \sigma_i \max_{y' \neq y} h(\mathbf{x}_i, y') \right]. \end{aligned}$$

Hạng thứ nhất được chặn bởi $\mathfrak{R}_m(\Pi_1(\mathcal{H}))$. Với hạng thứ hai, lớp hàm max gồm $k-1$ lớp tọa độ khác nhau. Áp dụng Bổ đề 8.1, ta có chặn

$$\frac{1}{m} \mathbb{E}_{S, \sigma} \left[\sup_{h \in \mathcal{H}} \sum_{i=1}^m \sigma_i \max_{y' \neq y} h(\mathbf{x}_i, y') \right] \leq (k-1) \mathfrak{R}_m(\Pi_1(\mathcal{H})).$$

Do đó, với mỗi y ,

$$\frac{1}{m} \mathbb{E}_{S, \sigma} \left[\sup_{h \in \mathcal{H}} \sum_{i=1}^m \sigma_i \rho_h(\mathbf{x}_i, y) \right] \leq k \mathfrak{R}_m(\Pi_1(\mathcal{H})).$$

Cuối cùng, vì phải cộng trên toàn bộ k nhãn $y \in \mathcal{Y}$, ta nhận được

$$\mathfrak{R}_m(\tilde{\mathcal{H}}) \leq k^2 \mathfrak{R}_m(\Pi_1(\mathcal{H})).$$

Thay chặn này vào kết quả của Bước 1:

$$R(h) \leq \widehat{R}_\rho(h) + 2 \cdot \frac{1}{\rho} \cdot k^2 \mathfrak{R}_m(\Pi_1(\mathcal{H})) + \sqrt{\frac{\log(1/\delta)}{2m}},$$

tức là

$$R(h) \leq \widehat{R}_\rho(h) + \frac{2k^2}{\rho} \mathfrak{R}_m(\Pi_1(\mathcal{H})) + \sqrt{\frac{\log(1/\delta)}{2m}}.$$

Định lý được chứng minh. $\square$

Giới hạn trên thể hiện đánh đổi quan trọng: chọn ρ lớn làm giảm hệ số $1/\rho$ trong thành phần độ phức tạp, nhưng lại khiến $\widehat{R}_\rho(h)$ có xu hướng tăng vì yêu cầu lề trên tập huấn luyện khắt khe hơn. Hệ số k^2 cho thấy khi số lớp tăng, bảo đảm lý thuyết yếu đi nhanh nếu không khai thác cấu trúc nhãn hoặc chiến lược giảm bài toán.

Hệ quả 8.1 cho giả thuyết kernel Xét lớp giả thuyết kernel $\mathcal{H}_{K,p}$ với

$$h(\mathbf{x}, y) = \langle \mathbf{w}_y, \Phi(\mathbf{x}) \rangle_{\mathcal{H}},$$

ma trận trọng số $W = (\mathbf{w}_1, \dots, \mathbf{w}_k)$ thỏa

$$\|W\|_{\mathcal{H},p} \leq \Lambda,$$

và kernel bị chặn

$$K(\mathbf{x}, \mathbf{x}) = \|\Phi(\mathbf{x})\|_{\mathcal{H}}^2 \leq r^2.$$

Khi đó Hệ quả 8.1 phát biểu rằng với xác suất ít nhất $1 - \delta$,

$$R(h) \leq \widehat{R}_\rho(h) + 2k^2 \sqrt{\frac{r^2 \Lambda^2}{m \rho^2}} + \sqrt{\frac{\log(1/\delta)}{2m}}.$$

Chứng minh. Hệ quả này thu được bằng cách kết hợp Định lý 8.1 với chặn Rademacher cho lớp tọa độ $\Pi_1(\mathcal{H}_{K,p})$. Ta chứng minh

$$\mathfrak{R}_m(\Pi_1(\mathcal{H}_{K,p})) \leq \sqrt{\frac{r^2 \Lambda^2}{m}}.$$

Bước 1: Định mức vector trọng số. Từ điều kiện $\|W\|_{\mathcal{H},p} \leq \Lambda$, suy ra chuẩn của từng vector trọng số riêng lẻ cũng bị chặn:

$$\|\mathbf{w}_y\|_{\mathcal{H}} \leq \Lambda, \quad \forall y \in [1, k].$$

Bước 2: Dùng bất đẳng thức Cauchy–Schwarz. Với một nhãn cố định y , độ

phức tạp Rademacher kinh nghiệm của lớp hàm $\mathbf{x} \mapsto \langle \mathbf{w}_y, \Phi(\mathbf{x}) \rangle$ thỏa

$$\begin{aligned}\widehat{\mathfrak{R}}_S(\Pi_1(\mathcal{H}_{K,p})) &= \frac{1}{m} \mathbb{E}_{\sigma} \left[\sup_{\|\mathbf{w}_y\|_{\mathcal{H}} \leq \Lambda} \sum_{i=1}^m \sigma_i \langle \mathbf{w}_y, \Phi(\mathbf{x}_i) \rangle_{\mathcal{H}} \right] \\ &= \frac{1}{m} \mathbb{E}_{\sigma} \left[\sup_{\|\mathbf{w}_y\|_{\mathcal{H}} \leq \Lambda} \left\langle \mathbf{w}_y, \sum_{i=1}^m \sigma_i \Phi(\mathbf{x}_i) \right\rangle_{\mathcal{H}} \right] \\ &\leq \frac{\Lambda}{m} \mathbb{E}_{\sigma} \left[\left\| \sum_{i=1}^m \sigma_i \Phi(\mathbf{x}_i) \right\|_{\mathcal{H}} \right].\end{aligned}$$

Bước 3: Dùng Jensen và khai triển bình phương. Theo bất đẳng thức Jensen,

$$\widehat{\mathfrak{R}}_S(\Pi_1(\mathcal{H}_{K,p})) \leq \frac{\Lambda}{m} \left(\mathbb{E}_{\sigma} \left[\left\| \sum_{i=1}^m \sigma_i \Phi(\mathbf{x}_i) \right\|_{\mathcal{H}}^2 \right] \right)^{1/2}.$$

Khai triển bình phương:

$$\begin{aligned}\mathbb{E}_{\sigma} \left[\left\| \sum_{i=1}^m \sigma_i \Phi(\mathbf{x}_i) \right\|_{\mathcal{H}}^2 \right] &= \mathbb{E}_{\sigma} \left[\sum_{i=1}^m \sum_{j=1}^m \sigma_i \sigma_j \langle \Phi(\mathbf{x}_i), \Phi(\mathbf{x}_j) \rangle_{\mathcal{H}} \right] \\ &= \sum_{i=1}^m \|\Phi(\mathbf{x}_i)\|_{\mathcal{H}}^2,\end{aligned}$$

nhờ $\mathbb{E}[\sigma_i \sigma_j] = 0$ khi $i \neq j$ và $\mathbb{E}[\sigma_i^2] = 1$.

Bước 4: Ráp giới hạn kernel. Do $K(\mathbf{x}_i, \mathbf{x}_i) = \|\Phi(\mathbf{x}_i)\|_{\mathcal{H}}^2 \leq r^2$, ta có

$$\widehat{\mathfrak{R}}_S(\Pi_1(\mathcal{H}_{K,p})) \leq \frac{\Lambda}{m} \left(\sum_{i=1}^m K(\mathbf{x}_i, \mathbf{x}_i) \right)^{1/2} \leq \frac{\Lambda}{m} \sqrt{m r^2} = \sqrt{\frac{r^2 \Lambda^2}{m}}.$$

Lấy kỳ vọng theo mẫu cho ta cùng chặn đối với $\mathfrak{R}_m(\Pi_1(\mathcal{H}_{K,p}))$. Thay trực tiếp vào Định lý 8.1:

$$R(h) \leq \widehat{R}_{\rho}(h) + \frac{2k^2}{\rho} \sqrt{\frac{r^2 \Lambda^2}{m}} + \sqrt{\frac{\log(1/\delta)}{2m}},$$

hay tương đương

$$R(h) \leq \widehat{R}_{\rho}(h) + 2k^2 \sqrt{\frac{r^2 \Lambda^2}{m \rho^2}} + \sqrt{\frac{\log(1/\delta)}{2m}}.$$

Hệ quả được chứng minh. □

4 Các thuật toán không kết hợp

Các thuật toán trong nhóm *không kết hợp* (uncombined algorithms) được thiết kế trực tiếp cho bài toán phân loại đa lớp, thay vì quy về nhiều bài toán phân loại nhị phân rồi ghép lại. Toàn bộ thông tin về các lớp được xử lý chung trong một bài toán tối ưu duy nhất, điều này đặt chúng vào vị trí đối lập với các thuật toán tổng hợp sẽ trình bày ở phần sau. Ba đại diện tiêu biểu gồm Multi-class SVM, Multi-class Boosting và Decision Trees.

4.1 Multi-class SVM

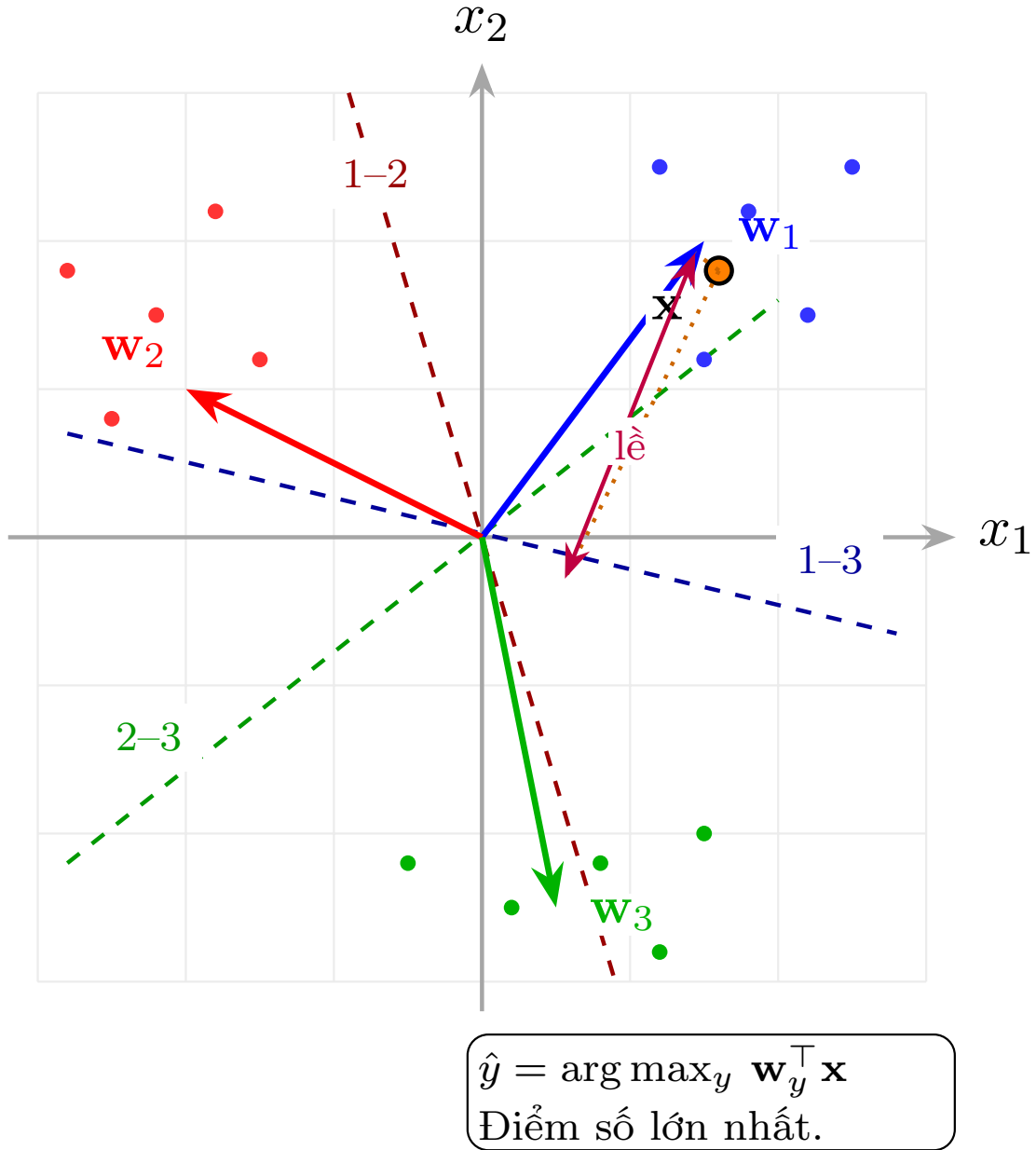
Multi-class SVM là mở rộng trực tiếp của SVM nhị phân dựa trên tư tưởng tối đa hóa lề. Thuật toán này liên hệ chặt chẽ với các giới hạn tổng quát hóa theo lề đã trình bày ở phần trước: nếu mô hình đạt lề lớn trên tập huấn luyện và norm của ma trận trọng số được kiểm soát, sai số tổng quát hóa có thể được chặn tốt hơn.

Thay vì huấn luyện nhiều mô hình nhị phân độc lập, Multi-class SVM học đồng thời k vector trọng số $\mathbf{w}_1, \mathbf{w}_2, \dots, \mathbf{w}_k$, mỗi vector tương ứng với một lớp. Với ánh xạ đặc trưng $\Phi(\mathbf{x})$, điểm số của lớp l được tính bởi

$$h(\mathbf{x}, l) = \langle \mathbf{w}_l, \Phi(\mathbf{x}) \rangle_{\mathcal{H}}.$$

Nhân dự đoán cho một điểm dữ liệu mới là lớp có điểm số lớn nhất:

$$g(\mathbf{x}) = \arg \max_{l \in \mathcal{Y}} \langle \mathbf{w}_l, \Phi(\mathbf{x}) \rangle_{\mathcal{H}}.$$



Hình 1: Minh họa ranh giới quyết định và lề trong Multi-class SVM.

Xây dựng bài toán tối ưu: Xuất phát từ cận tổng quát hóa dựa trên margin đã thiết lập, ta có thể rút ra một thuật toán học đa lớp cụ thể bằng cách tối thiểu hóa vế phải của bất đẳng thức đó. Với mỗi mẫu huấn luyện $(\mathbf{x}_i, y_i)$, ta định nghĩa biến bù (slack variable):

$$\xi_i = \max(1 - [\mathbf{w}_{y_i} \cdot \Phi(\mathbf{x}_i) - \max_{y \neq y_i} \mathbf{w}_y \cdot \Phi(\mathbf{x}_i)], 0), \quad \forall i \in [m].$$

$\xi_i > 0$ khi điểm $\mathbf{x}_i$ bị phân loại sai hoặc đúng nhưng với margin nhỏ hơn 1. Bài toán tối ưu SVM đa lớp khi đó được phát biểu như sau:

$$\min_{W, \xi} \quad \frac{1}{2} \sum_{l=1}^k \|\mathbf{w}_l\|_{\mathcal{H}}^2 + C \sum_{i=1}^m \xi_i \quad (1)$$

$$\text{s.t.} \quad \langle \mathbf{w}_{y_i}, \Phi(\mathbf{x}_i) \rangle_{\mathcal{H}} \geq \langle \mathbf{w}_l, \Phi(\mathbf{x}_i) \rangle_{\mathcal{H}} + 1 - \xi_i, \quad \forall i \in [m], \forall l \in \mathcal{Y} \setminus \{y_i\},$$

và $\xi_i \geq 0$, $i = 1, \dots, m$.

Ở đây $W = (\mathbf{w}_1, \dots, \mathbf{w}_k)$ là ma trận các vector trọng số, $C > 0$ là tham số điều chỉnh sự đánh đổi giữa độ phức tạp của mô hình và sai số huấn luyện.

Ý nghĩa hình học. Ràng buộc trong (1) yêu cầu điểm số (score) của lớp đúng y_i phải cao hơn điểm số của *bất kỳ* lớp sai l một khoảng ít nhất bằng $1 - \xi_i$. Khi $\xi_i = 0$, mẫu nằm đúng phía và đủ xa khỏi ranh giới quyết định; khi $\xi_i > 0$, ràng buộc bị vi phạm và mô hình bị phạt tỉ lệ với $C\xi_i$.

Liên hệ với hàm mất mát hinge đa lớp: Với mỗi mẫu $(\mathbf{x}_i, y_i)$, hàm mất mát hinge đa lớp có dạng

$$\ell_i(W) = \max_{l \neq y_i} [1 + \langle \mathbf{w}_l, \Phi(\mathbf{x}_i) \rangle - \langle \mathbf{w}_{y_i}, \Phi(\mathbf{x}_i) \rangle]_+.$$

Biến slack ξ_i trong bài toán primal chính là một chặn trên cho mất mát này. Vì thế bài toán SVM có thể hiểu là tối thiểu hóa regularized empirical risk:

$$\frac{1}{2} \|W\|_{\mathcal{H},2}^2 + C \sum_{i=1}^m \ell_i(W).$$

Bài toán đối ngẫu: Bằng cách lập Lagrangian và áp dụng điều kiện KKT cho (1), bài toán primal có thể được chuyển sang bài toán đối ngẫu thuần túy theo kernel K :

$$\max_{\alpha \in \mathbb{R}^{m \times k}} \quad \sum_{i=1}^m \alpha_i \cdot e_{y_i} - \frac{1}{2} \sum_{i,j=1}^m (\alpha_i \cdot \alpha_j) K(\mathbf{x}_i, \mathbf{x}_j) \quad (2)$$

$$\text{s.t.} \quad 0 \leq \alpha_i \leq C, \quad \alpha_i \cdot \mathbf{1} = 0, \quad \forall i \in [m],$$

trong đó $\alpha \in \mathbb{R}^{m \times k}$ là ma trận nhân tử Lagrange, α_i là hàng thứ i của α , và e_l là vector đơn vị thứ l trong $\mathbb{R}^k$. Nhờ đó, Multi-class SVM có thể dùng kernel trick để học biên quyết định phi tuyến mà không cần biểu diễn tường minh $\Phi(\mathbf{x})$.

Nhận xét về độ phức tạp: Cả hai bài toán nguyên thủy và đối ngẫu đều là bài toán *Quy hoạch Toàn phương* (Quadratic Program – QP), tổng quát hóa trực tiếp SVM nhị phân. Tuy nhiên, số ràng buộc tỉ lệ với $m(k-1) = \Omega(mk)$. Khi k lớn, bài toán có thể rất tốn kém về mặt tính toán. Trong thực tiễn, người ta thường áp dụng các chiến lược phân rã (decomposition) hoặc sử dụng bộ giải tối ưu chuyên biệt để giảm tải.

Lưu ý

Multi-class SVM là thuật toán “không kết hợp” vì toàn bộ k lớp được tối ưu trong cùng một bài toán duy nhất. Mô hình không cần huấn luyện k bài toán One-vs-All hay $k(k-1)/2$ bài toán One-vs-One.

4.2 Boosting đa lớp – AdaBoost.MH

Tóm tắt ký hiệu riêng: Trong AdaBoost.MH, $\mathcal{Y} = \{-1, +1\}^k$ là không gian nhãn đa nhãn; h_t là bộ phân loại yếu tại vòng t ; α_t là trọng số của h_t ; $g_n = \sum_{t=1}^n \alpha_t h_t$ là mô hình cộng gộp sau n vòng; $F(\alpha)$ là hàm mục tiêu cần cực tiểu hóa; và D_t là phân phối xác suất/trọng số trên các cặp mẫu–nhãn tại vòng t .

Boosting là nhóm thuật toán xây dựng mô hình mạnh bằng cách kết hợp nhiều bộ học yếu. Trong bối cảnh đa lớp và đa nhãn, một biến thể quan trọng là AdaBoost.MH, được giới thiệu bởi Schapire và Singer. Thuật toán này được thiết kế cho dữ liệu có không gian nhãn

$$\mathcal{Y} = \{-1, +1\}^k,$$

trong đó mỗi mẫu $\mathbf{x}_i$ có vector nhãn $\mathbf{y}_i = (y_i[1], y_i[2], \dots, y_i[k])$ với $y_i[l] \in \{-1, +1\}$. Giá trị $y_i[l] = +1$ nghĩa là mẫu i thuộc nhãn l , còn $y_i[l] = -1$ nghĩa là không thuộc nhãn l . AdaBoost.MH trả về một tổ hợp lỗi của các bộ phân loại cơ sở:

$$g_n(\mathbf{x}, l) = \sum_{t=1}^n \alpha_t h_t(\mathbf{x}, l).$$

Hàm mục tiêu: AdaBoost.MH tối ưu một hàm mục tiêu lỗi, khả vi, đóng vai trò là chặn trên của hàm mất mát đa lớp–đa nhãn [1]:

$$F(\alpha) = \sum_{i=1}^m \sum_{l=1}^k \exp(-y_i[l] g_n(\mathbf{x}_i, l)). \quad (3)$$

Đây là cận trên vì với mọi x và l ta có $\mathbf{1}_{y[l] \neq g(x,l)} \leq e^{-y[l]g(x,l)}$. Nếu $y_i[l]g_n(\mathbf{x}_i, l)$ lớn và dương, số hạng mũ nhỏ. Ngược lại, nếu dự đoán sai hoặc lẻ âm, số hạng mũ tăng mạnh và thuật toán sẽ tập trung hơn vào cặp mẫu–nhãn này.

Cơ chế hoạt động: AdaBoost.MH có thể được xem như áp dụng *coordinate descent* lên hàm $F(\alpha)$. Tương đương về mặt tính toán, thuật toán có thể được nhìn như chạy AdaBoost nhị phân chuẩn trên tập dữ liệu đã mở rộng S' , trong đó mỗi điểm $(\mathbf{x}_i, \mathbf{y}_i)$ được tách thành k mẫu:

$$(\mathbf{x}_i, \mathbf{y}_i) \longrightarrow ((\mathbf{x}_i, 1), y_i[1]), \dots, ((\mathbf{x}_i, k), y_i[k]).$$

Tập S' gồm mk mẫu, và hàm mục tiêu (3) trùng khớp với hàm mục tiêu của AdaBoost trên S' . Nhờ đó, toàn bộ lý thuyết phân tích hội tụ và cận tổng quát hóa đã thiết lập cho AdaBoost nhị phân đều áp dụng được tại đây [1].

[MỞ RỘNG] Điều kiện học yếu và độ phức tạp.

Điều kiện học yếu (weak learning condition) yêu cầu ở mỗi vòng lặp tồn tại $h_t : \mathcal{X} \times \mathcal{Y} \rightarrow \{-1, +1\}$ sao cho $\Pr_{(i,l) \sim D_t}[h_t(\mathbf{x}_i, l) \neq y_i[l]] < 1/2$. Điều kiện này có thể khó đảm bảo khi các lớp gần nhau hoặc khi dữ liệu mất cân bằng nghiêm trọng.

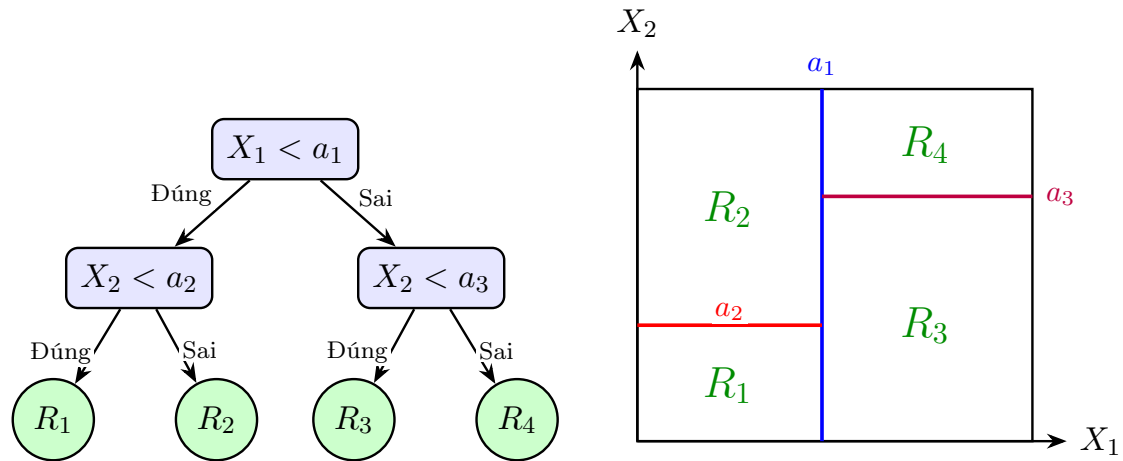
Về độ phức tạp: với $\mathcal{X} \subseteq \mathbb{R}^N$ và boosting stumps làm bộ phân loại cơ sở, thuật toán có độ phức tạp $\mathcal{O}((mk) \log(mk) + mkNT)$. Khi k lớn, chi phí tăng theo mk làm thuật toán khó mở rộng cho bài toán extreme multi-label.

4.3 Cây quyết định (Decision Trees)

Tóm tắt ký hiệu riêng: Với cây quyết định, n là một nút trong cây, q là câu hỏi phân tách tại nút, $p_l(n)$ là tỷ lệ mẫu lớp l rơi vào nút n , $F(n)$ là độ vẩn đục của nút, $\tilde{F}(n, q)$ là mức giảm vẩn đục sau khi tách nút n bằng câu hỏi q , và λ là tham số điều chuẩn khi cắt tỉa cây.

Cây quyết định nhị phân là cấu trúc cây biểu diễn một phân hoạch của không gian đặc trưng $\mathcal{X}$. Mỗi nút trong (internal node) tương ứng với một câu hỏi về đặc trưng; mỗi lá (leaf) đại diện cho một vùng phân hoạch độc lập không giao nhau, được gán nhãn $l \in \mathcal{Y}$ bằng cách lấy đa số (majority vote) trên các điểm huấn luyện rơi vào vùng đó.

Kiến trúc cây quyết định Phân hoạch không gian đặc trưng



Hình 2: Đối sánh giữa cấu trúc cây quyết định và phân hoạch không gian đặc trưng.

Hàm quyết định: Tại một nút lá n , nhãn dự đoán cho toàn bộ vùng tương ứng thường được chọn bằng bầu chọn đa số:

$$g_n = \arg \max_{l \in \mathcal{Y}} \#\{i : \mathbf{x}_i \in n, y_i = l\}.$$

Nếu cần xác suất, cây có thể ước lượng phân phối nhãn tại lá bằng tỷ lệ mẫu:

$$\hat{p}_n(l) = \frac{\#\{i : \mathbf{x}_i \in n, y_i = l\}}{\#\{i : \mathbf{x}_i \in n\}}.$$

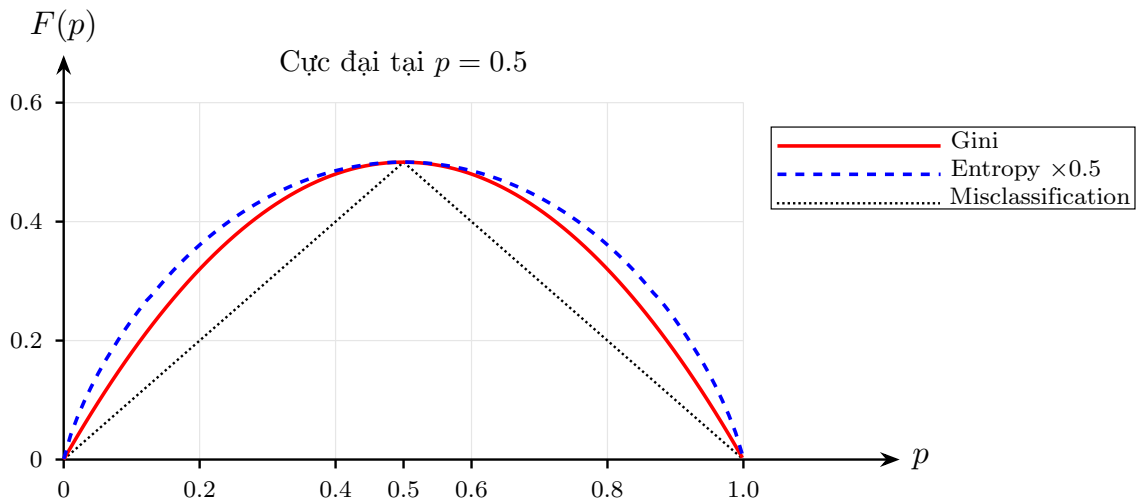
Học cây – Chiến lược tham lam: Bài toán tìm cây quyết định có sai số nhỏ nhất là NP-khó. Vì vậy, cây thường được xây dựng bằng *chiến lược tham lam*: tại mỗi nút, thuật toán xét nhiều phép chia ứng viên và chọn phép chia q làm giảm độ vẩn đục (node impurity decrease) $\tilde{F}(n, q)$ nhiều nhất:

$$\tilde{F}(n, q) = F(n) - \frac{|n_L|}{|n|} F(n_L) - \frac{|n_R|}{|n|} F(n_R).$$

Cho $p_l(n)$ là tỷ lệ mẫu thuộc lớp l tại nút n . Ba tiêu chí độ đo impurity phổ biến là:

$$F(n) = \begin{cases} 1 - \max_{l \in [k]} p_l(n) & (\text{Misclassification}), \\ -\sum_{l=1}^k p_l(n) \log_2 p_l(n) & (\text{Entropy}), \\ \sum_{l=1}^k p_l(n) (1 - p_l(n)) & (\text{Gini index}). \end{cases} \quad (4)$$

Entropy và Gini lồi chặt và khả vi, đảm bảo luôn giảm impurity nghiêm ngặt – vì vậy được ưa dùng hơn trong thực tiễn.



Hình 3: So sánh trực quan các hàm đo độ vẩn đục thường dùng trong cây quyết định.

Chiến lược chống overfitting: grow-then-prune: Cây quyết định rất dễ overfit. Một chiến lược phổ biến là “trồng rồi tỉa” (grow-then-prune): đầu tiên phát triển một cây lớn, sau đó tìm cây con tối ưu bằng cách cực tiểu hóa hàm chi phí có điều chuẩn:

$$G_\lambda(\text{tree}') = \sum_{n \in \text{leaves}(\text{tree}')} |n| F(n) + \lambda |\text{leaves}(\text{tree}')|, \quad (5)$$

với $\lambda \geq 0$ là tham số điều chỉnh được chọn qua cross-validation.

Ưu và nhược điểm: Cây quyết định nhanh về huấn luyện và dự đoán, dễ diễn giải. Tuy nhiên, nó không ổn định: thay đổi nhỏ trong dữ liệu có thể tạo ra cây hoàn toàn khác.

[Mở rộng]

Chứng minh chiều VC của cây quyết định

Xét họ giả thuyết $\mathcal{T}$ gồm các cây quyết định nhị phân có n nút trong (internal nodes) trên không gian đặc trưng N chiều. Mỗi nút trong sử dụng một phép chia dạng

$$X_j \leq a,$$

trong đó $j \in \{1, \dots, N\}$ là chỉ số đặc trưng và a là ngưỡng chia. Ta chứng minh rằng

$$\text{VCdim}(\mathcal{T}) = O(n \log(nN)),$$

và khi $N \gg n$, chặn này thường được viết gọn là $O(n \log N)$.

Bước 1: Đếm số cấu trúc cây. Một cây nhị phân có n nút trong có số dạng hình học (topologies) không vượt quá số Catalan thứ n :

$$C_n = \frac{1}{n+1} \binom{2n}{n} \leq 4^n.$$

Do đó, số cách chọn cấu trúc cây bị chặn trên bởi 4^n .

Bước 2: Đếm số phép chia tại mỗi nút. Với m điểm dữ liệu cố định, tại mỗi nút ta có N lựa chọn đặc trưng. Sau khi chọn một đặc trưng X_j , các điểm dữ liệu khi chiếu lên trục đó tạo ra nhiều nhất m ngưỡng phân biệt về mặt nhãn tách. Vì vậy, tại mỗi nút có nhiều nhất Nm phép chia khác nhau trên tập m điểm. Với n nút trong, số cách gán phép chia cho một cấu trúc cây cố định bị chặn bởi

$$(Nm)^n.$$

Bước 3: Chặn hàm tăng trưởng. Kết hợp hai bước trên, số dichotomies tối đa mà họ cây có thể tạo ra trên m điểm thỏa

$$\Pi_{\mathcal{T}}(m) \leq 4^n (Nm)^n = (4Nm)^n.$$

Nếu m điểm bị shatter, ta phải có $\Pi_{\mathcal{T}}(m) = 2^m$, nên

$$2^m \leq (4Nm)^n.$$

Lấy logarit cơ số 2 hai vế:

$$m \leq n \log_2(4Nm) = n \log_2(4N) + n \log_2 m. \quad (*)$$

Bước 4: Suy ra chặn VC-dimension. Dùng bổ đề chuẩn: nếu $m \leq a + b \log_2 m$ với $a, b > 0$, thì $m = O(a + b \log b)$. Áp dụng vào (*) với

$$a = n \log_2(4N), \quad b = n,$$

ta được

$$m = O(n \log N + n \log n) = O(n \log(nN)).$$

Vì vậy,

$$\text{VCdim}(\mathcal{T}) = O(n \log(nN)).$$

Trong các bài toán chiều rất cao, chẳng hạn xử lý ngôn ngữ tự nhiên với $N \gg n$, thành phần $\log N$ chi phối, nên có thể viết gọn chặn trên là

$$\text{VCdim}(\mathcal{T}) = O(n \log N).$$

5 Nhóm thuật toán kết hợp

5.1 Động lực

Thuật toán kết hợp (aggregated algorithms) giải quyết phân loại đa lớp bằng cách quy bài toán ban đầu về nhiều bài toán nhị phân, sau đó tổng hợp đầu ra của các bộ phân loại con. Cách tiếp cận này tận dụng được các thuật toán nhị phân đã phát triển mạnh, đồng thời linh hoạt hơn khi thay đổi bộ học nền [2].

5.2 One-vs-All

One-vs-All (OVA), còn gọi là One-vs-Rest, huấn luyện k bộ phân loại nhị phân. Với lớp l , bộ phân loại h_l xem các mẫu thuộc lớp l là dương và tất cả mẫu còn lại là âm:

$$y_i^{(l)} = \begin{cases} +1, & y_i = l, \\ -1, & y_i \neq l. \end{cases}$$

Sau khi huấn luyện, mỗi mô hình trả về điểm số $f_l(\mathbf{x})$. Quy tắc dự đoán là

$$h_{\text{OVA}}(\mathbf{x}) = \arg \max_{l \in \mathcal{Y}} f_l(\mathbf{x}).$$

Algorithm 1: Huấn luyện One-vs-All

Input: Tập huấn luyện $S = \{(\mathbf{x}_i, y_i)\}_{i=1}^m$, tập nhãn $\mathcal{Y}$

Output: k bộ phân loại nhị phân $f_1, \dots, f_k$

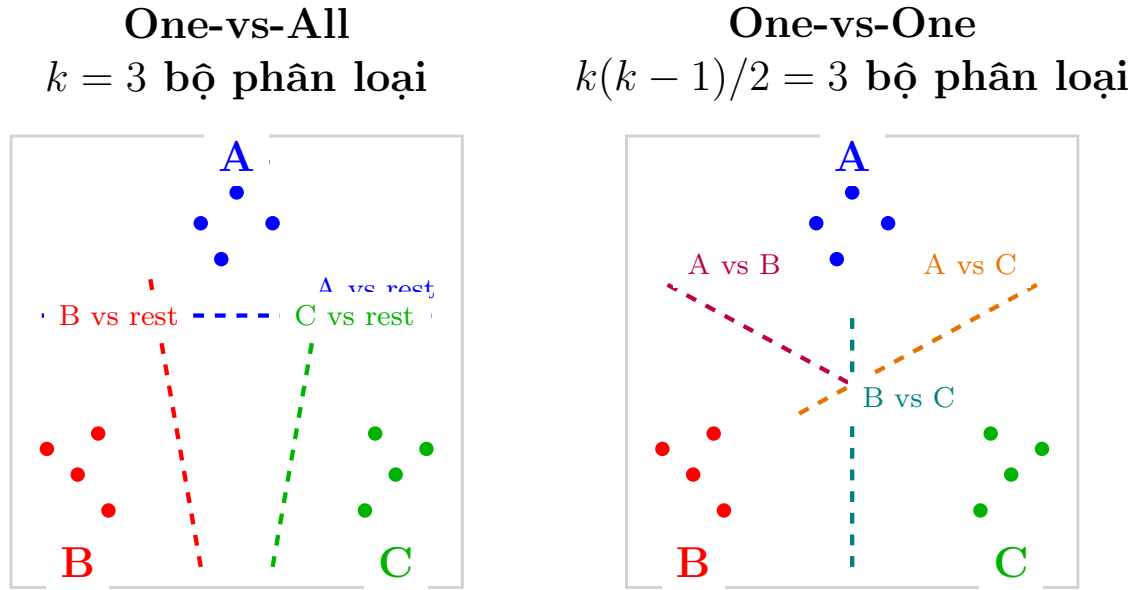
for $l = 1$ **to** k **do**

Tạo nhãn nhị phân $y_i^{(l)} = +1$ nếu $y_i = l$, ngược lại -1 ;
Huấn luyện bộ phân loại f_l trên $\{(\mathbf{x}_i, y_i^{(l)})\}_{i=1}^m$;

Ví dụ với bốn lớp Táo, Cam, Chuối và Nho, OVA huấn luyện bốn mô hình: Táo-vs-còn lại, Cam-vs-còn lại, Chuối-vs-còn lại và Nho-vs-còn lại. Nếu một mẫu mới nhận điểm số lần lượt $(0.2, 0.8, 0.1, 0.4)$ thì mô hình dự đoán lớp Cam.

Ưu điểm của OVA là đơn giản, dễ cài đặt và chỉ cần k mô hình. Chi phí dự đoán là $O(kc_t)$ nếu c_t là chi phí đánh giá một bộ phân loại nhị phân. Tuy nhiên, mỗi bài toán nhị phân thường mất cân bằng nghiêm trọng vì một lớp phải đối đầu với toàn bộ các lớp còn lại. Ngoài ra, điểm số giữa các bộ phân loại độc lập có thể không được hiệu chỉnh cùng thang đo, dẫn đến trường hợp nhiều mô hình cùng tự tin hoặc tất cả đều kém tự tin.

Để giảm hạn chế này, có thể dùng class weight, hiệu chỉnh xác suất bằng Platt scaling/isotonic regression, hoặc chọn ngưỡng riêng cho từng lớp trên validation set.



Hình 4: Minh họa cách OVA và OVO tạo các ranh giới nhị phân khác nhau trong bài toán đa lớp.

5.3 One-vs-One

One-vs-One (OVO) huấn luyện một bộ phân loại cho mỗi cặp lớp, do đó cần

$$\binom{k}{2} = \frac{k(k-1)}{2}$$

mô hình nhị phân. Bộ phân loại $h_{ll'}$ chỉ dùng các mẫu thuộc hai lớp l và l' . Khi dự đoán, mỗi mô hình bỏ phiếu cho một trong hai lớp và lớp có nhiều phiếu nhất được chọn:

$$h_{\text{OVO}}(\mathbf{x}) = \arg \max_{l \in \mathcal{Y}} V_l(\mathbf{x}).$$

Với $k = 4$ lớp trái cây gồm Táo, Cam, Chuối và Nho, OVO cần sáu bộ phân loại. Bảng dưới minh họa các bài toán nhị phân tương ứng.

Bảng 10: Ví dụ các bộ phân loại OVO cho bốn lớp

Cặp lớp	Bộ phân loại	Dữ liệu huấn luyện
Táo–Cam	h_{12}	Chỉ mẫu Táo và Cam
Táo–Chuối	h_{13}	Chỉ mẫu Táo và Chuối
Táo–Nho	h_{14}	Chỉ mẫu Táo và Nho
Cam–Chuối	h_{23}	Chỉ mẫu Cam và Chuối
Cam–Nho	h_{24}	Chỉ mẫu Cam và Nho
Chuối–Nho	h_{34}	Chỉ mẫu Chuối và Nho

Giả sử các trận đấu cho số phiếu lần lượt là Táo: 1, Cam: 3, Chuối: 0, Nho: 2. Khi đó

$$h_{\text{OVO}}(\mathbf{x}) = \arg \max\{1, 3, 0, 2\} = 2,$$

và mẫu được dự đoán là Cam.

Mệnh đề 5.1 (Chặn lỗi OVO). *Giả sử lỗi tổng quát hóa của mọi bộ phân loại cặp không vượt quá ϵ . Khi đó lỗi của bộ phân loại OVO thỏa*

$$R(h_{\text{OVO}}) \leq (k - 1)\epsilon.$$

Chứng minh. Nếu OVO phân loại sai một mẫu có nhãn đúng l^* , thì lớp l^* không thể thắng tất cả các trận đấu với các lớp còn lại. Vì vậy tồn tại ít nhất một bộ phân loại cặp liên quan đến l^* dự đoán sai. Áp dụng union bound trên $k - 1$ cặp liên quan đến l^* cho ta kết quả. $\square$

[Mở rộng]

Chứng minh tính chặt của giới hạn lỗi OVO

Mệnh đề trên cho thấy nếu mọi bộ phân loại nhị phân trong OVO có sai số tổng quát hóa không vượt quá ϵ , thì sai số của hệ OVO không vượt quá $(k - 1)\epsilon$. Chặn này xuất phát từ bất đẳng thức Boole (union bound):

$$\mathbb{P}\left(\bigcup_{j=2}^k A_j\right) \leq \sum_{j=2}^k \mathbb{P}(A_j),$$

trong đó A_j là biến cố bộ phân loại liên quan đến lớp đúng và lớp j dự đoán sai. Union bound đạt dấu bằng khi các biến cố A_j xung khắc từng đôi một. Ta xây dựng một ví dụ để cho thấy chặn $(k - 1)\epsilon$ là chặt.

Bước 1: Thiết lập phân phối. Xét bài toán có k lớp $\mathcal{Y} = \{1, 2, \dots, k\}$ và giả sử mọi điểm dữ liệu đều có nhãn đúng là lớp 1, tức

$$\mathbb{P}(y = 1) = 1.$$

Phân hoạch không gian đặc trưng $\mathcal{X}$ thành k vùng rời nhau

$$\mathcal{X} = \bigcup_{j=1}^k R_j, \quad R_i \cap R_j = \emptyset \quad (i \neq j),$$

với xác suất

$$\mathbb{P}(\mathbf{x} \in R_j) = \epsilon, \quad j = 2, \dots, k,$$

và

$$\mathbb{P}(\mathbf{x} \in R_1) = 1 - (k - 1)\epsilon.$$

Điều kiện này hợp lệ khi $(k - 1)\epsilon \leq 1$.

Bước 2: Định nghĩa các bộ phân loại cặp. Với mỗi bộ phân loại h_{1j} phân biệt lớp đúng 1 và lớp sai j ($j \geq 2$), đặt

$$h_{1j}(\mathbf{x}) = \begin{cases} j, & \mathbf{x} \in R_j, \\ 1, & \mathbf{x} \notin R_j. \end{cases}$$

Vì nhãn đúng luôn là 1, bộ h_{1j} chỉ sai trên vùng R_j . Do đó

$$R(h_{1j}) = \mathbb{P}(\mathbf{x} \in R_j) = \epsilon.$$

Với các bộ phân loại giữa hai lớp sai $a, b \geq 2$, định nghĩa sao cho nếu $\mathbf{x} \in R_a$ thì bộ phân loại bầu cho a , còn nếu $\mathbf{x} \in R_b$ thì bầu cho b . Cách định nghĩa ngoài hai vùng này có thể chọn tùy ý vì không ảnh hưởng đến lập luận chính.

Bước 3: Phân tích biểu quyết OVO. Nếu $\mathbf{x} \in R_1$, mọi bộ h_{1j} đều bầu cho lớp 1. Khi đó lớp 1 nhận $k - 1$ phiếu và OVO dự đoán đúng.

Nếu $\mathbf{x} \in R_j$ với $j \geq 2$, bộ h_{1j} bầu cho lớp j . Ngoài ra, với mọi $l \notin \{1, j\}$, bộ h_{lj} cũng bầu cho lớp j theo thiết kế trên. Vì vậy lớp j nhận tổng cộng

$$1 + (k - 2) = k - 1$$

phiếu và thắng biểu quyết. Tuy nhiên, nhãn đúng vẫn là lớp 1, nên OVO dự đoán sai trên toàn bộ vùng R_j .

Bước 4: Tính sai số OVO. Hệ OVO chỉ sai trên hợp của các vùng $R_2, \dots, R_k$. Vì các vùng này rời nhau từng đôi một,

$$R(h_{\text{OVO}}) = \mathbb{P}\left(\bigcup_{j=2}^k R_j\right) = \sum_{j=2}^k \mathbb{P}(\mathbf{x} \in R_j) = \sum_{j=2}^k \epsilon = (k - 1)\epsilon.$$

Do đó tồn tại một phân phối và một họ bộ phân loại cặp sao cho mỗi bộ có sai số đúng bằng ϵ , trong khi sai số của toàn hệ OVO đạt đúng $(k - 1)\epsilon$. Vì vậy chặn trong Mệnh đề chặn lỗi OVO là chặt; không thể cải thiện chặn này nếu không bổ sung giả định mạnh hơn về phân phối dữ liệu hoặc tương quan giữa các lỗi cặp.

Về chi phí, OVA cần $O(k)$ mô hình và dự đoán nhanh hơn. OVO cần $O(k^2)$ mô hình, nhưng mỗi mô hình được huấn luyện trên tập con nhỏ hơn. Với thuật toán nhị

phân có chi phí $O(n^\alpha)$ và dữ liệu cân bằng, chi phí huấn luyện OVO là

$$T_{\text{OVO}}^{\text{train}} = \binom{k}{2} O\left(\left(\frac{2m}{k}\right)^\alpha\right) = O(k^{2-\alpha}m^\alpha),$$

trong khi OVA có chi phí

$$T_{\text{OVA}}^{\text{train}} = kO(m^\alpha) = O(km^\alpha).$$

Nếu $\alpha = 2$, $k = 10$ và $m = 1000$, OVA cần xấp xỉ $10 \times 1000^2 = 10^7$ đơn vị tính toán, còn OVO cần $45 \times 200^2 = 1.8 \times 10^6$ đơn vị. Vì vậy OVO có thể thuận lợi khi thuật toán nền có chi phí siêu tuyến tính.

Nhược điểm của OVO là chi phí lưu trữ và dự đoán tăng bậc hai theo số lớp. Ngoài ra, khi nhiều lớp nhận cùng số phiếu cao nhất, hệ thống cần cơ chế phá hòa, chẳng hạn chọn lớp có tổng margin lớn nhất hoặc dùng xác suất cặp để biểu quyết có trọng số.

5.3.1 Biểu quyết có trọng số

Thay vì mỗi bộ phân loại cặp chỉ đóng góp một phiếu, ta có thể gán trọng số theo độ tin cậy:

$$h_{\text{OVO-weighted}}(\mathbf{x}) = \arg \max_{l \in \mathcal{Y}} \sum_{l' \neq l} w_{ll'}(\mathbf{x}) \mathbb{I}[h_{ll'}(\mathbf{x}) \text{ bầu cho } l].$$

Một lựa chọn phổ biến là $w_{ll'}(\mathbf{x}) = |f_{ll'}(\mathbf{x})|$, tức mô hình có margin lớn hơn được tin tưởng hơn. Nếu có xác suất cặp $p_{ll'}(\mathbf{x})$, ta có thể cộng các xác suất thay vì cộng phiếu cứng.

5.4 Error-Correcting Output Codes

Error-Correcting Output Codes (ECOC) tổng quát hóa OVA và OVO bằng cách gán mỗi lớp một mã nhị phân [2]. Gọi ma trận mã là

$$M \in \{-1, +1\}^{k \times c},$$

trong đó hàng M_l là mã của lớp l , còn cột j định nghĩa một bài toán nhị phân bằng cách chia các lớp thành nhóm dương và âm.

Với mỗi cột j , ta tạo bài toán nhị phân bằng cách gán nhãn

$$\tilde{y}_i^{(j)} = M_{y_i, j} \in \{-1, +1\}.$$

Sau đó huấn luyện bộ phân loại $h_j : \mathcal{X} \rightarrow \{-1, +1\}$. Quy trình huấn luyện ECOC gồm ba bước: tái gán nhãn theo từng cột của ma trận mã, huấn luyện c bộ phân loại nhị

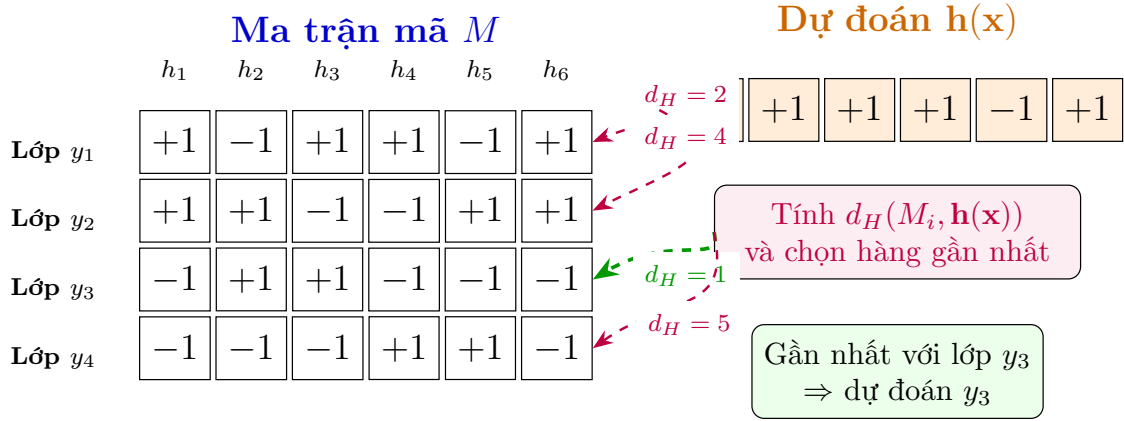
phân và giải mã dự đoán bằng khoảng cách Hamming:

$$h_{\text{ECOC}}(\mathbf{x}) = \arg \min_{l \in \mathcal{Y}} d_H(M_l, \mathbf{h}(\mathbf{x})).$$

Khoảng cách Hamming giữa hai vector nhị phân có thể viết là

$$d_H(\mathbf{u}, \mathbf{v}) = \frac{1}{2} \sum_{j=1}^c (1 - u_j v_j).$$

ECOC có khả năng sửa lỗi khi các codeword cách xa nhau. Nếu khoảng cách Hamming tối thiểu giữa hai mã là $d_{\min}$, hệ thống có thể sửa tối đa $\lfloor (d_{\min} - 1)/2 \rfloor$ lỗi bit trong điều kiện lý tưởng.



Hình 5: Minh họa ma trận mã ECOC và quy trình giải mã bằng khoảng cách Hamming.

Bảng 11: So sánh nhanh OVA, OVO và ECOC

Phương pháp	Số mô hình	Ưu điểm	Hạn chế
OVA	k	Đơn giản, dự đoán nhanh	Mất cân bằng lớp
OVO	$k(k - 1)/2$	Mỗi bài toán nhỏ hơn	Lưu trữ/dự đoán tốn hơn
ECOC	c	Có khả năng sửa lỗi	Phụ thuộc thiết kế mã

Có thể xem OVA và OVO là các trường hợp đặc biệt của ECOC. Với OVA, mỗi cột của ma trận mã tương ứng một lớp đối đầu với phần còn lại. Với OVO, mỗi cột chỉ phân biệt một cặp lớp, còn các lớp không thuộc cặp có thể được bỏ qua trong biến thể mã tam phân $\{-1, 0, +1\}$.

5.5 Dự đoán có cấu trúc (Structured Prediction)

Trong nhiều ứng dụng thực tế như xử lý ngôn ngữ tự nhiên, thị giác máy tính hoặc sinh học tính toán, đầu ra của mô hình không chỉ là một nhãn đơn lẻ mà còn có cấu

trúc nội tại phức tạp, chẳng hạn chuỗi, cây hoặc đồ thị. Ví dụ điển hình là gán nhãn từ loại (part-of-speech tagging), trong đó một câu gồm nhiều từ cần được gán một chuỗi nhãn tương ứng. Bài toán này có thể xem là một dạng tổng quát hóa của phân loại đa lớp, nhưng có ba thách thức riêng biệt:

- **Không gian nhãn lớn theo cấp số nhân:** nếu một câu có độ dài n và tập nhãn từ loại là Σ , số chuỗi nhãn khả dĩ là $|\Sigma|^n$.
- **Sự phụ thuộc giữa các cấu trúc con:** các thành phần trong đầu ra không độc lập; một số chuỗi nhãn có thể không phù hợp về mặt ngữ pháp hoặc ngữ cảnh.
- **Hàm mất mát phức tạp:** hàm mất mát $L(y', y)$ giữa nhãn dự đoán y' và nhãn đúng y thường không phải mất mát 0–1, mà phụ thuộc vào từng cấu trúc con, chẳng hạn khoảng cách Hamming giữa hai chuỗi nhãn.

Hàm giả thuyết và đặc trưng chung. Trong dự đoán có cấu trúc, đặc trưng thường phụ thuộc đồng thời vào đầu vào $\mathbf{x}$ và đầu ra y . Do đó, ta định nghĩa ánh xạ đặc trưng kết hợp

$$\Phi(\mathbf{x}, y).$$

Lớp giả thuyết thường dùng có dạng tuyến tính theo đặc trưng chung:

$$h(\mathbf{x}) = \arg \max_{y \in \mathcal{Y}} \mathbf{w}^\top \Phi(\mathbf{x}, y),$$

trong đó $\mathbf{w}$ là vector trọng số cần học. Điểm khác biệt so với phân loại đa lớp thông thường là phép cực đại phải tìm trên không gian $\mathcal{Y}$ có cấu trúc và thường rất lớn.

Maximum Margin Markov Networks (M3N). M3N đưa hàm mất mát vào lễ bằng cơ chế phạt cộng dồn (additive penalization). Với mỗi mẫu $(\mathbf{x}_i, y_i)$, một nhãn sai $y \neq y_i$ bị xem là vi phạm nghiêm trọng hơn nếu mất mát $L(y_i, y)$ lớn hơn. Một dạng mục tiêu có thể viết là

$$\min_{\mathbf{w}} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^m \max_{y \neq y_i} [L(y_i, y) - \mathbf{w}^\top \Delta \Phi_i(y)]_+,$$

trong đó

$$\Delta \Phi_i(y) = \Phi(\mathbf{x}_i, y_i) - \Phi(\mathbf{x}_i, y), \quad [z]_+ = \max(0, z).$$

Khi cấu trúc nhãn có tính Markov, chẳng hạn chuỗi hoặc cây, và hàm mất mát có thể phân rã theo các thành phần cục bộ, bài toán suy luận trong bước tối ưu có thể được giải hiệu quả bằng quy hoạch động hoặc các thuật toán truyền tin trên đồ thị. Thiết lập này cũng kết hợp tự nhiên với các kernel trên cấu trúc rời rạc, chẳng hạn PDS kernel, khi cần đo độ tương đồng giữa các đối tượng có cấu trúc.

SVMStruct. SVMStruct sử dụng tư tưởng lề tối đa nhưng kết hợp mất mát theo cơ chế phạt nhân (multiplicative combination). Một dạng mục tiêu tương ứng là

$$\min_{\mathbf{w}} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^m \max_{y \neq y_i} L(y_i, y) [1 - \mathbf{w}^\top \Delta \Phi_i(y)]_+.$$

Do số lượng nhãn cấu trúc có thể rất lớn, số ràng buộc tiềm năng trong bài toán tối ưu cũng rất lớn. Vì vậy, SVMStruct thường được giải bằng chiến lược cutting-plane: ở mỗi vòng lặp, thuật toán tìm nhãn vi phạm mạnh nhất (most violating constraint) rồi thêm ràng buộc tương ứng vào tập đang xét.

Conditional Random Fields (CRFs). CRF là mô hình phổ biến cho dữ liệu chuỗi. Về mặt hình thức, mục tiêu của CRF gần với M3N nhưng thay toán tử cực đại cứng bằng hàm xấp xỉ trơn dạng log-sum-exp. Một dạng mục tiêu có điều chuẩn là

$$\min_{\mathbf{w}} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^m \log \sum_{y \in \mathcal{Y}} \exp (L(y_i, y) - \mathbf{w}^\top \Delta \Phi_i(y)).$$

Cách làm này cho phép tối ưu trơn hơn so với hard-max, đồng thời vẫn khai thác được cấu trúc chuỗi thông qua các thuật toán suy luận như forward-backward hoặc Viterbi trong trường hợp phù hợp.

Lưu ý

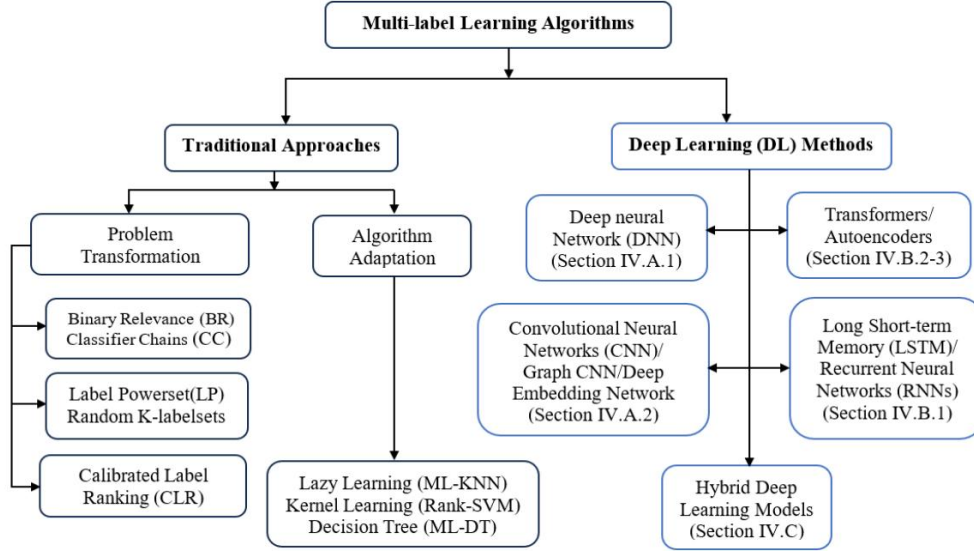
Dự đoán có cấu trúc mở rộng phân loại đa lớp từ việc chọn một nhãn đơn sang việc chọn một đối tượng có cấu trúc. Trọng tâm của nhóm phương pháp này là thiết kế đặc trưng chung $\Phi(\mathbf{x}, y)$, hàm mất mát cấu trúc $L(y', y)$ và thuật toán suy luận hiệu quả trên không gian đầu ra rất lớn.

6 Ứng dụng Deep Learning cho bài toán Multi-class / Multi-label

Các phần trước đã trình bày cơ sở lý thuyết và các thuật toán nền tảng cho phân loại đa lớp/đa nhãn, gồm Multi-class SVM, AdaBoost.MH, Decision Tree, One-vs-All, One-vs-One, ECOC và dự đoán có cấu trúc. Các phương pháp này có ưu điểm là rõ ràng về mặt mô hình hóa và dễ phân tích. Tuy nhiên, khi dữ liệu đầu vào là văn bản dài, ảnh, âm thanh hoặc dữ liệu y sinh, việc thiết kế đặc trưng thủ công như TF-IDF hay vector thống kê đơn giản thường không đủ để biểu diễn ngữ nghĩa sâu của dữ liệu.

Deep Learning được xem như một hướng mở rộng tự nhiên vì mô hình có thể học biểu diễn trực tiếp từ dữ liệu đầu vào, đồng thời học chung nhiều đầu ra trong cùng một kiến trúc. Với bài toán multi-label, điều này đặc biệt quan trọng vì các nhãn

thường không độc lập hoàn toàn: một ảnh có thể đồng thời chứa người, xe và đường phố; một văn bản có thể vừa thuộc chủ đề thể thao vừa thuộc chủ đề sức khỏe. Vì vậy, trọng tâm của phần này là chỉ ra cách mạng nơ-ron được điều chỉnh để xử lý đầu ra đa lớp/đa nhãn.



Hình 6: Phân loại tổng quát các hướng tiếp cận trong multi-label learning: nhóm truyền thống và nhóm Deep Learning.

6.1 Đầu ra và hàm mất mát trong mạng nơ-ron

Ở các phần trước, bài toán phân loại đa lớp đã được mô tả thông qua hàm điểm số $h(x, y)$ và quy tắc chọn nhãn có điểm số lớn nhất. Trong mạng nơ-ron, tầng cuối thường sinh ra một vector điểm số chưa chuẩn hóa, gọi là *logits*. Cách chuyển logits thành dự đoán phụ thuộc vào việc bài toán là đơn nhãn hay đa nhãn.

Với bài toán multi-class đơn nhãn, mỗi mẫu chỉ thuộc đúng một lớp trong $\{1, 2, \dots, k\}$. Vì các lớp loại trừ nhau, tầng đầu ra thường dùng softmax:

$$p(y = l \mid x) = \frac{\exp(z_l)}{\sum_{j=1}^k \exp(z_j)}, \quad l = 1, \dots, k.$$

Khi đó, các xác suất trên k lớp có tổng bằng 1 và mô hình dự đoán lớp có xác suất lớn nhất:

$$\hat{y} = \arg \max_l p(y = l \mid x).$$

Hàm mất mát thường dùng là cross-entropy đa lớp:

$$\mathcal{L}_{CE} = -\frac{1}{m} \sum_{i=1}^m \log p(y_i \mid x_i).$$

Với bài toán multi-label, một mẫu có thể đồng thời mang nhiều nhãn. Do đó, các nhãn không được chuẩn hóa cạnh tranh như softmax. Nếu có L nhãn, tầng đầu ra thường gồm L nút sigmoid:

$$p_{il} = \sigma(z_{il}) = \frac{1}{1 + \exp(-z_{il})}, \quad l = 1, \dots, L.$$

Mỗi p_{il} biểu diễn xác suất mẫu x_i mang nhãn l . Nhãn dự đoán thường được xác định bằng một ngưỡng τ_l :

$$\hat{y}_{il} = \begin{cases} 1, & p_{il} \geq \tau_l, \\ 0, & p_{il} < \tau_l. \end{cases}$$

Hàm mất mát phổ biến là binary cross-entropy theo từng nhãn:

$$\mathcal{L}_{BCE} = -\frac{1}{m} \sum_{i=1}^m \sum_{l=1}^L [y_{il} \log(p_{il}) + (1 - y_{il}) \log(1 - p_{il})].$$

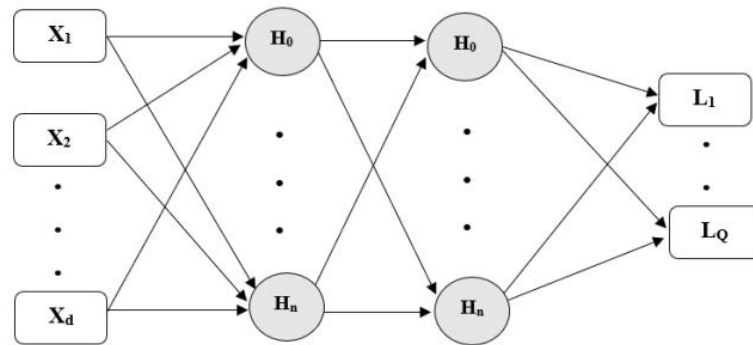
Lưu ý rằng đầu ra sigmoid chỉ xem mỗi nhãn như một quyết định nhị phân ở tầng cuối. Điều này không có nghĩa toàn bộ mô hình học các nhãn hoàn toàn tách rời. Trong mạng nơ-ron, các tầng ẩn phía trước vẫn được học chung, nên mô hình có thể chia sẻ biểu diễn giữa những nhãn có liên quan. Nếu muốn khai thác quan hệ nhãn rõ hơn, các kiến trúc như attention, RNN/LSTM, Transformer hoặc graph thường được đặt ở phần encoder trước khi đưa ra dự đoán cuối cùng.

Thiết lập	Tầng đầu ra và loss	Ý nghĩa dự đoán
Multi-class đơn nhãn	Softmax trên k lớp, thường dùng cross-entropy đa lớp.	Các lớp loại trừ nhau. Mỗi mẫu được gán đúng một nhãn bằng quy tắc $\hat{y} = \arg \max_l p_l$.
Multi-label cơ bản	Sigmoid riêng cho từng nhãn, thường dùng binary cross-entropy theo từng nhãn.	Mỗi nhãn là một quyết định nhị phân. Một mẫu có thể đồng thời nhận nhiều nhãn nếu $p_l \geq \tau_l$.
Multi-label có mô hình hóa quan hệ nhãn	Dùng encoder chung, attention, RNN/LSTM, Transformer hoặc graph trước tầng sigmoid/ranking.	Mô hình không chỉ học đặc trưng đầu vào mà còn khai thác quan hệ đồng xuất hiện, thứ tự hoặc cấu trúc giữa các nhãn.

Bảng 12: So sánh tầng đầu ra của mạng nơ-ron trong bài toán multi-class và multi-label.

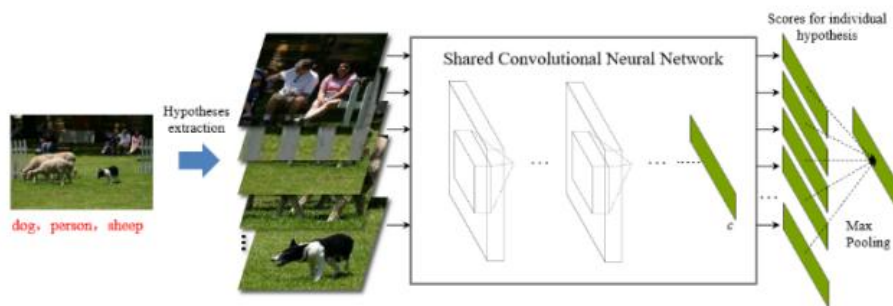
6.2 Các kiến trúc học sâu tiêu biểu

Mạng truyền thẳng và BP-MLL. Cách mở rộng đơn giản nhất là dùng một mạng nhiều tầng với q nút đầu ra, mỗi nút tương ứng với một nhãn. Mô hình BP-MLL là một trong những hướng đầu tiên áp dụng mạng nơ-ron cho multi-label classification. Thay vì chỉ đo sai số từng nhãn riêng lẻ, BP-MLL tối ưu sao cho điểm của các nhãn đúng cao hơn điểm của các nhãn sai. Cách làm này phù hợp để minh họa bước chuyển từ các bộ phân loại nhị phân độc lập sang một mô hình học chung nhiều đầu ra.



Hình 7: Mạng nơ-ron truyền thẳng cho multi-label classification: các đặc trưng đầu vào được ánh xạ qua tầng ẩn đến nhiều nút đầu ra, mỗi nút tương ứng với một nhãn.

CNN và Graph CNN. CNN phù hợp với dữ liệu có cấu trúc cục bộ như ảnh, văn bản ngắn hoặc tín hiệu. Trong ảnh đa nhãn, một ảnh có thể chứa nhiều đối tượng cùng lúc, nên CNN thường được dùng làm bộ trích xuất đặc trưng, sau đó tầng đầu ra sigmoid dự đoán xác suất cho từng nhãn. Một số mô hình mở rộng dùng Graph CNN, trong đó mỗi nhãn được xem như một nút trong đồ thị; cạnh giữa các nút biểu diễn quan hệ đồng xuất hiện hoặc quan hệ ngữ nghĩa giữa các nhãn. Cách này giúp mô hình hóa tốt hơn các phụ thuộc nhãn mà CNN thông thường chưa xử lý trực tiếp.

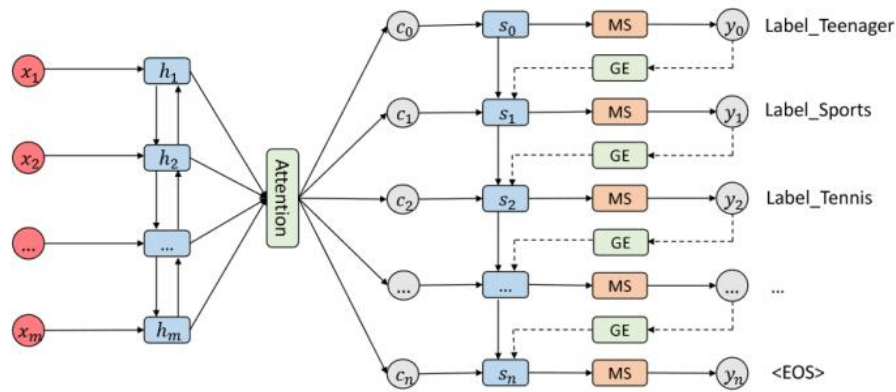


Hình 8: Minh họa hướng dùng CNN cho phân loại ảnh đa nhãn: các vùng/giả thuyết đối tượng được đưa qua CNN dùng chung, sau đó tổng hợp điểm để dự đoán nhiều nhãn.

RNN/LSTM, Transformer và mô hình lai. RNN và LSTM phù hợp với dữ liệu tuần tự như văn bản, chuỗi thời gian hoặc tín hiệu y sinh. Trong multi-label classification, LSTM có thể được dùng để sinh chuỗi nhãn, trong đó nhãn dự đoán trước ảnh hưởng đến nhãn dự đoán sau. Ý tưởng này gần với Classifier Chains, nhưng thay vì dùng nhiều bộ phân loại rời rạc, mô hình học phụ thuộc nhãn trong một kiến trúc tuần tự duy nhất.

$$p(y | x) = \prod_{t=1}^n p(y_t | y_1, y_2, \dots, y_{t-1}, x).$$

Transformer mở rộng hướng này bằng cơ chế attention, cho phép mô hình tập trung vào những phần khác nhau của đầu vào khi dự đoán từng nhãn. Trong xử lý văn bản, các mô hình dựa trên BERT thường cho biểu diễn ngữ cảnh tốt hơn CNN/RNN truyền thống. Trong thị giác máy tính, Transformer hoặc các mô hình lai CNN–Transformer có thể học đồng thời quan hệ không gian giữa vùng ảnh và quan hệ ngữ nghĩa giữa nhãn.



Hình 9: Mô hình sinh chuỗi nhãn dùng RNN/LSTM: nhãn ở bước sau được dự đoán dựa trên đầu vào và các nhãn đã sinh trước đó.

6.3 Liên hệ với các phương pháp truyền thống

Deep Learning không thay thế hoàn toàn SVM, Decision Tree, OVA hay ECOC trong mọi trường hợp. Khi dữ liệu nhỏ, đặc trưng rõ ràng và yêu cầu diễn giải cao, các mô hình truyền thống vẫn phù hợp. Tuy nhiên, trong những bài toán có dữ liệu phi cấu trúc, quan hệ nhãn phức tạp hoặc số nhãn lớn, mạng nơ-ron thường linh hoạt hơn vì vừa học đặc trưng đầu vào, vừa học biểu diễn dùng chung cho nhiều nhãn.

Hướng tiếp cận	Vai trò	Giới hạn
SVM, OVA	Nền tảng tuyến tính/kernel, dễ phân tích bằng lẽ và độ phức tạp.	Khó học quan hệ nhãn nếu các bộ phân loại được học độc lập.
Decision Tree	Dễ diễn giải, dự đoán nhanh.	Dễ overfit và không ổn định khi dữ liệu thay đổi.
AdaBoost.MH	Xử lý trực tiếp các cặp mẫu–nhãn trong thiết lập đa nhãn.	Chi phí tăng theo số mẫu nhãn với số nhãn.
DNN/ LSTM/ Transformer	CNN/ Học biểu diễn đầu vào và biểu diễn nhãn trong cùng một mô hình.	Cần nhiều dữ liệu, tốn tài nguyên và khó giải thích hơn.

Bảng 13: Liên hệ giữa các phương pháp truyền thống và học sâu trong phân loại đa lớp/đa nhãn.

6.4 Nhận xét tổng quan

Vai trò của Deep Learning trong bài toán multi-class/multi-label thể hiện ở hai khía cạnh chính. Thứ nhất, mạng nơ-ron có khả năng tự học biểu diễn đặc trưng từ dữ liệu, thay vì phụ thuộc nhiều vào đặc trưng thủ công như các phương pháp truyền thống. Thứ hai, trong bài toán đa nhãn, các tầng ẩn dùng chung cho phép mô hình chia sẻ thông tin giữa các nhãn; khi kết hợp thêm attention, RNN/LSTM, Transformer hoặc đồ thị nhãn, mô hình có thể khai thác tốt hơn quan hệ đồng xuất hiện, thứ tự hoặc cấu trúc giữa các nhãn.

Tuy nhiên, học sâu không loại bỏ hoàn toàn các khó khăn của bài toán. Các mô hình này thường cần nhiều dữ liệu huấn luyện, chi phí tính toán cao, khó diễn giải hơn so với cây quyết định hoặc mô hình tuyến tính, và vẫn gặp trở ngại khi số lượng nhãn tăng lên quy mô rất lớn. Vì vậy, trong các bài toán extreme multi-label, Deep Learning thường cần được kết hợp với các kỹ thuật giảm không gian nhãn, tạo shortlist hoặc tổ chức nhãn theo cây để đảm bảo khả năng mở rộng.

7 Các nghiên cứu tiên tiến trong phân loại đa lớp cực đại

7.1 Tổng quan Extreme Multi-label Classification

Trong thực tế, khi số lượng nhãn tăng lên quy mô hàng trăm nghìn hoặc hàng triệu, các thuật toán nền tảng như Multi-class SVM, One-versus-All (OVA) hoặc One-versus-One (OVO) gặp giới hạn nghiêm trọng về chi phí tính toán. Nếu số nhãn là L , việc đánh giá trực tiếp hàm điểm cho toàn bộ nhãn thường có độ phức tạp $O(L)$ cho mỗi mẫu, chưa kể chi phí huấn luyện và lưu trữ mô hình.

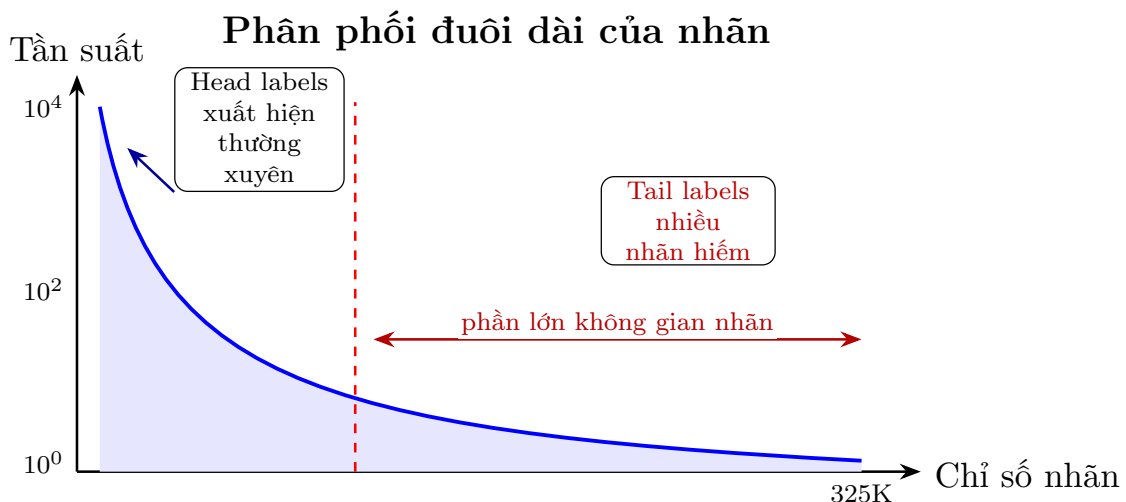
Extreme Multi-label Classification (XMC) được nghiên cứu nhằm giải quyết tình huống này. Trong XMC, mỗi mẫu có thể gắn với nhiều nhãn, nhưng số nhãn đúng

thường rất nhỏ so với tổng số nhãn. Vì vậy, ma trận nhãn có dạng rất thưa và phân phối nhãn thường có tính đuôi dài: một số ít nhãn xuất hiện thường xuyên, trong khi phần lớn nhãn chỉ xuất hiện ở số lượng mẫu hạn chế.

Các thách thức chính của XMC gồm:

- **Chi phí tính toán:** đánh giá toàn bộ L nhãn là không khả thi khi L rất lớn.
- **Nhãn đuôi:** các nhãn hiếm khó học vì có ít ví dụ huấn luyện.
- **Mất cân bằng dữ liệu:** mô hình dễ thiên về các nhãn phổ biến nếu chỉ tối ưu loss thông thường.
- **Biểu diễn ngữ nghĩa:** Bag-of-Words hoặc TF-IDF có thể không đủ để nắm bắt ngữ cảnh sâu của văn bản.
- **Triển khai thực tế:** hệ thống cần dự đoán nhanh, cập nhật nhãn mới và xử lý văn bản dài.

Phần này phân tích một số hướng nghiên cứu tiên tiến giai đoạn 2019–2024, tập trung vào cách các phương pháp hiện đại kế thừa, mở rộng và khắc phục giới hạn của lý thuyết nền tảng về phân loại đa lớp.



Hình 10: Phân phối đuôi dài của nhãn trong bài toán phân loại đa nhãn cực lớn.

7.2 Phân tích các bài báo tiêu biểu

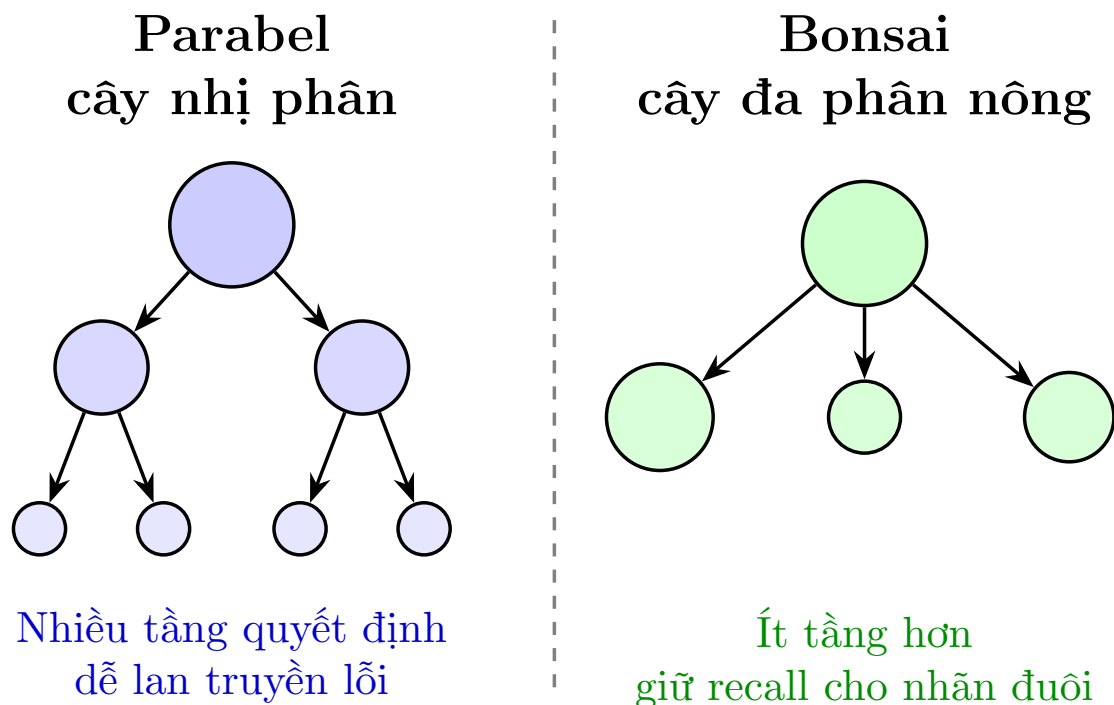
7.2.1 Bonsai: Diverse and shallow trees for XMC

Bài toán đặt ra. Bonsai là một phương pháp cây nông đa dạng cho XMC [8]. Các phương pháp dựa trên cây trước đây, chẳng hạn Parabel, thường ép cây phải phân chia nhị phân và cân bằng. Ràng buộc này làm cây trở nên sâu khi số nhãn lớn. Cây càng sâu thì lỗi ở các nút gần gốc càng dễ lan truyền xuống các tầng dưới, gây hiện tượng

error propagation. Một khi mẫu đi nhầm nhánh, các nhãn đúng ở nhánh khác có thể không bao giờ được xét đến. Điều này đặc biệt bất lợi với các nhãn đuôi hiếm gặp.

Đóng góp chính. Bonsai nói lỏng ràng buộc cân bằng và cho phép phân chia đa phân (K-way partitioning), trong đó K có thể rất lớn, ví dụ $K \geq 100$. Thay vì ép mỗi nút chia thành hai nhánh cân bằng, Bonsai dùng phân cụm như K-means để tạo ra các cây nông và đa dạng. Cây nông giúp giảm số bước quyết định từ gốc đến lá, nhờ đó giảm nguy cơ lỗi lan truyền. Tính đa dạng của nhiều cây giúp cải thiện khả năng bao phủ nhãn đuôi.

Parabel vs. Bonsai trong phân hoạch nhãn



Hình 11: Ý tưởng cây nông đa phân trong Bonsai nhằm giảm lỗi lan truyền.

Liên hệ với lý thuyết nền tảng. Bonsai phát triển trực tiếp từ tư tưởng cây quyết định đã trình bày trong nhóm thuật toán không kết hợp. Tuy nhiên, thay vì phân hoạch không gian đặc trưng bằng greedy split dựa trên node impurity như Gini hoặc Entropy, Bonsai phân hoạch **không gian nhãn**. Việc phân hoạch có thể dựa trên biểu diễn đầu vào, biểu diễn đầu ra hoặc biểu diễn kết hợp. Như vậy, Bonsai giữ lại ưu điểm tính toán của cây nhưng điều chỉnh cấu trúc cây để phù hợp với bài toán XMC.

Nhận xét. Ưu điểm của Bonsai là giảm chiều sâu cây, tăng khả năng bảo toàn nhãn đuôi và cải thiện tốc độ dự đoán. Hạn chế là mô hình vẫn có thể chịu lỗi lan truyền ở

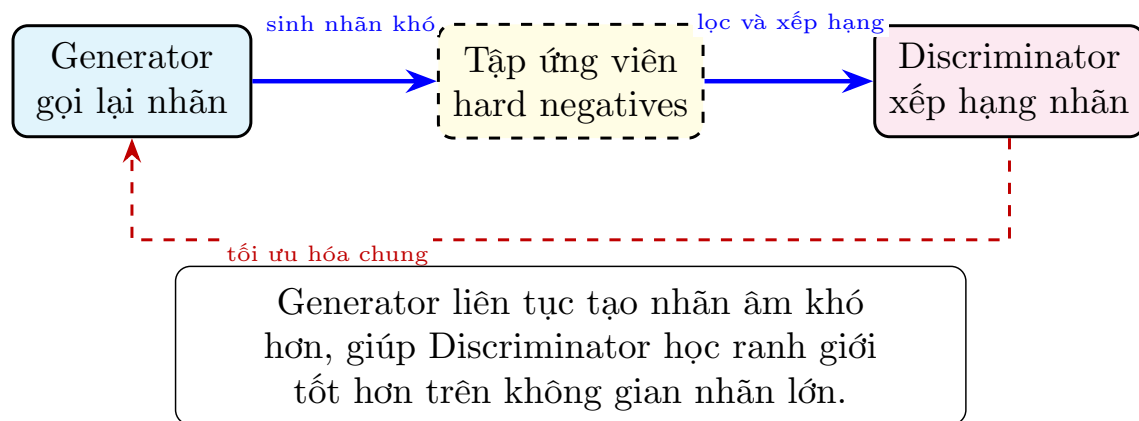
các nút cao; ngoài ra chất lượng phân cụm nhãn ảnh hưởng mạnh đến chất lượng dự đoán cuối cùng.

7.2.2 LightXML: Transformer with Dynamic Negative Sampling

Bài toán đặt ra. Các mô hình học sâu cho XMC thường phải xử lý số nhãn âm cực lớn. Nếu dùng static negative sampling, mô hình chỉ nhìn thấy một nhóm nhãn âm cố định hoặc ít thay đổi, dẫn đến nguy cơ overfit vào các nhãn âm cụ thể và khó học được ranh giới tốt giữa nhãn đúng và nhãn sai khó. Ngoài ra, huấn luyện Transformer trên không gian nhãn cực lớn đòi hỏi tài nguyên tính toán rất cao.

Đóng góp chính. LightXML sử dụng mạng hợp tác sinh–phân biệt (generative cooperative networks). Bộ tạo (Generator) có nhiệm vụ gọi lại một tập nhãn ứng viên, đóng vai trò như cơ chế lấy mẫu tiêu cực động. Ở giai đoạn đầu, Generator có thể cung cấp các nhãn âm dễ; khi mô hình tốt hơn, nó sinh ra các nhãn âm khó hơn. Bộ phân biệt (Discriminator) sau đó xếp hạng các nhãn trong tập ứng viên này.

LightXML: lấy mẫu tiêu cực động



Hình 12: Cơ chế lấy mẫu tiêu cực động trong LightXML với Generator và Discriminator.

Liên hệ với lý thuyết nền tảng. Về bản chất, phần xếp hạng của LightXML vẫn tương đồng với chiến lược OVA: mỗi nhãn được xem như một bài toán nhị phân độc lập và dùng loss nhị phân cho từng nhãn. Điểm khác biệt là LightXML không đánh giá toàn bộ L nhãn. Generator thu gọn không gian nhãn từ $O(L)$ xuống một shortlist nhỏ, giúp giảm đáng kể chi phí tính toán.

Nhận xét. LightXML kế thừa sức mạnh biểu diễn của Transformer và giải quyết nút thắt nhãn âm bằng dynamic negative sampling. Tuy nhiên, phương pháp vẫn phụ thuộc

vào giới hạn chiều dài đầu vào của Transformer và chất lượng shortlist do Generator tạo ra.

7.2.3 XR-Transformer: Fast Multi-Resolution Fine-tuning

Bài toán đặt ra. XR-Transformer đề xuất fine-tuning Transformer đa độ phân giải cho XMC [10]. Fine-tuning trực tiếp các mô hình Transformer lớn như BERT hoặc RoBERTa trên hàng trăm nghìn đến hàng triệu nhãn là rất tốn thời gian. Nếu mỗi bước phải tính điểm cho toàn bộ nhãn, độ phức tạp tuyến tính theo L khiến huấn luyện có thể kéo dài nhiều ngày hoặc nhiều tuần.

Đóng góp chính. XR-Transformer chia bài toán thành nhiều độ phân giải thông qua cây nhãn phân cấp (Hierarchical Label Tree – HLT). Mô hình được fine-tune theo chiến lược coarse-to-fine: trước hết học ở mức cụm nhãn thô, sau đó tinh chỉnh dần xuống các cụm nhỏ hơn và cuối cùng đến nhãn cụ thể. Ngoài ra, để bù đắp thông tin bị mất do Transformer phải cắt cụt văn bản, XR-Transformer kết hợp đặc trưng ngữ nghĩa từ Transformer với đặc trưng thưa TF-IDF.

Liên hệ với lý thuyết nền tảng. XR-Transformer giải quyết trực tiếp thách thức chi phí tính toán trong phân loại đa lớp cực đại. Bằng cách biến phân loại phẳng trên L nhãn thành phân loại theo cây, phương pháp giảm số nhãn cần xét ở mỗi bước. Trong điều kiện cây được tổ chức tốt, chi phí dự đoán có thể giảm từ $O(L)$ xuống gần $O(\log L)$ hoặc phụ thuộc vào kích thước shortlist ở từng tầng.

Nhận xét. Điểm mạnh của XR-Transformer là kết hợp ba thành phần: Transformer để học ngữ nghĩa, TF-IDF để giữ tín hiệu thống kê thưa, và cây nhãn để tăng tốc. Hạn chế là chất lượng cây nhãn ảnh hưởng lớn đến kết quả; nếu mẫu bị đưa sai nhánh ở tầng thô, nhãn đúng có thể bị loại khỏi các bước tinh hơn.

7.2.4 ICXML: In-Context Learning for Zero-shot XMC

Bài toán đặt ra. ICXML là khung in-context learning cho zero-shot XMC [11]. Trong nhiều hệ thống thực tế, nhãn mới có thể xuất hiện mà không có dữ liệu huấn luyện đi kèm. Đây là thiết lập zero-shot XMC. Một cách tự nhiên là dùng mô hình ngôn ngữ lớn (LLM), nhưng không thể đưa hàng trăm nghìn hoặc hàng triệu nhãn vào prompt vì giới hạn độ dài ngữ cảnh.

Đóng góp chính. ICXML đề xuất khung gồm hai giai đoạn: Generate và Rerank. Ở giai đoạn Generate, LLM tạo các pseudo-demonstrations hoặc tín hiệu ngữ nghĩa để sinh ra danh sách nhãn rút gọn. Ở giai đoạn Rerank, LLM thực hiện phân loại đa

nhân trên shortlist này bằng in-context learning. Nhờ đó, hệ thống không cần huấn luyện lại toàn bộ mô hình cho các nhãn mới.

Liên hệ với lý thuyết nền tảng. ICXML khác với giả định cổ điển của phân loại có giám sát, trong đó mô hình học tham số từ tập huấn luyện có nhãn $(\mathbf{x}_i, y_i)$. Thay vì học các vector trọng số $\mathbf{w}_l$ như Multi-class SVM hoặc OVA, ICXML dựa vào tri thức nền và khả năng suy luận trong ngữ cảnh của LLM. Tuy nhiên, nó vẫn cần cơ chế rút gọn nhãn vì không gian nhãn đầy đủ quá lớn.

Nhận xét. ICXML mở ra hướng zero-shot cho XMC, đặc biệt hữu ích khi nhãn thay đổi nhanh. Hạn chế là chi phí suy luận của LLM cao, kết quả phụ thuộc mạnh vào prompt, và mô hình có thể sinh hoặc chọn nhãn không nhất quán nếu shortlist không được kiểm soát tốt.

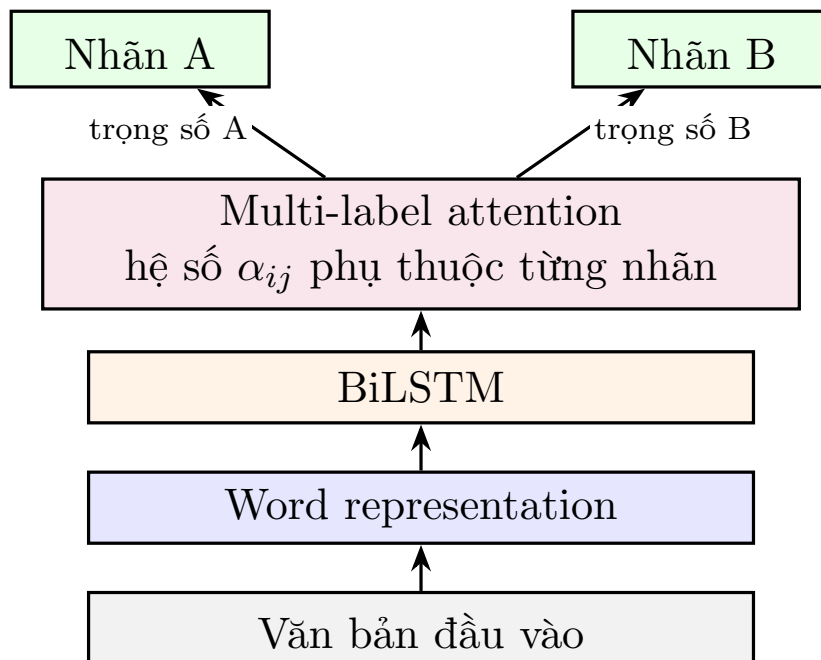
7.3 Đánh giá xu hướng phát triển

Sự dịch chuyển từ lý thuyết phân loại đa lớp nền tảng sang XMC hiện đại thể hiện qua ba xu hướng chính.

7.3.1 Từ đặc trưng thừa sang ngữ nghĩa sâu

Các thuật toán truyền thống thường dựa trên Bag-of-Words hoặc TF-IDF. Những đặc trưng này có ưu điểm là thưa, dễ tính và mở rộng tốt, nhưng khó nắm bắt ngữ cảnh dài, đồng nghĩa và quan hệ ngữ nghĩa phức tạp. Các phương pháp hiện đại chuyển sang dùng BiLSTM, attention và Transformer để học biểu diễn ngữ nghĩa sâu. AttentionXML dùng BiLSTM và attention đa nhãn [9]; LightXML và XR-Transformer dùng Transformer để học biểu diễn văn bản theo ngữ cảnh.

AttentionXML: attention riêng cho từng nhãn



Hình 13: Cơ chế attention đa nhãn trong AttentionXML.

Tuy vậy, đặc trưng thừa chưa biến mất hoàn toàn. XR-Transformer cho thấy TF-IDF vẫn hữu ích khi kết hợp với Transformer, đặc biệt vì TF-IDF giữ lại tín hiệu từ toàn bộ văn bản trong khi Transformer thường bị giới hạn độ dài đầu vào.

7.3.2 Cấu trúc cây vẫn là cốt lõi của tính toán

Dù mô hình học sâu có năng lực biểu diễn cao hơn, việc đánh giá hàm điểm trên hàng triệu nhãn vẫn là điểm nghẽn tính toán. Vì vậy, cấu trúc cây tiếp tục đóng vai trò trung tâm. Bonsai dùng cây nông đa dạng để giảm lỗi lan truyền; AttentionXML kết hợp attention với cây nhãn xác suất; XR-Transformer dùng cây nhãn phân cấp để fine-tune theo nhiều độ phân giải.

Điểm chung là các phương pháp này không cố tính điểm cho mọi nhãn. Thay vào đó, chúng dùng cây để tạo shortlist hoặc dẫn đường quá trình dự đoán, từ đó đưa độ phức tạp từ tuyến tính theo L về mức gần logarit hoặc theo kích thước tập ứng viên nhỏ.

7.3.3 Tập trung vào phân phối đuôi dài

Vấn đề mất cân bằng lớp trong lý thuyết nền tảng trở nên nghiêm trọng hơn nhiều trong XMC. Phần lớn nhãn là tail labels, có rất ít mẫu huấn luyện. Nếu chỉ tối ưu các thước đo thông thường, mô hình dễ đạt kết quả tốt trên nhãn phổ biến nhưng bỏ qua nhãn hiếm.

Do đó, các nghiên cứu XMC thường dùng các thước đo như propensity-scored precision@k (PSP@k) hoặc propensity-scored nDCG. Ý tưởng là tăng trọng số đánh giá cho nhãn hiếm để phản ánh đúng giá trị của việc dự đoán tail labels. Xu hướng này cho thấy bài toán XMC không chỉ là bài toán tăng tốc tính toán, mà còn là bài toán học trong phân phối mất cân bằng cực đoan.

[Mở rộng]

7.4 Hạn chế của các phương pháp hiện tại và đề xuất cải tiến

7.4.1 Hạn chế hiện tại

Giới hạn độ dài văn bản của Transformer. Các mô hình như XR-Transformer và LightXML thường phải cắt đầu vào xuống 128 hoặc 512 token để tránh vượt bộ nhớ GPU. Nguyên nhân là cơ chế self-attention truyền thống có độ phức tạp $O(N^2)$ theo độ dài chuỗi N . Với các tài liệu dài như văn bản pháp luật, hồ sơ y khoa hoặc bài báo khoa học, việc cắt cụt làm mất nhiều thông tin ngữ nghĩa quan trọng.

Lỗi lan truyền trong cây. Mặc dù Bonsai dùng cây nông để giảm error propagation, sai số ở các nút gần gốc vẫn có thể khiến mẫu đi sai nhánh. Khi đó, nhãn đúng có thể bị loại khỏi quá trình xét về sau. Đây là hạn chế chung của các phương pháp dựa trên cây, đặc biệt khi mỗi bước chỉ giữ lại một số ít nhánh ứng viên.

Phụ thuộc vào shortlist. Nhiều hệ thống hiện đại dùng bước tạo shortlist trước khi rerank. Nếu shortlist không chứa nhãn đúng, bộ reranker có năng lực biểu diễn cao cũng không thể phục hồi kết quả. Vì vậy, recall của bước tạo ứng viên là yếu tố then chốt trong XMC.

7.4.2 Đề xuất cải tiến

Sử dụng kiến trúc có độ phức tạp tuyến tính. Một hướng cải tiến là thay Transformer truyền thống bằng các kiến trúc có độ phức tạp gần tuyến tính theo độ dài chuỗi, chẳng hạn Longformer, State Space Models hoặc Mamba. Các mô hình này cho phép xử lý văn bản dài hàng nghìn token mà không cần cắt cụt quá mạnh, từ đó giữ được nhiều thông tin hơn cho các nhãn đuôi.

Kết hợp XR-Transformer với đồ thị tri thức. XR-Transformer hiện bù đắp thông tin mất do cắt cụt bằng cách nối vector Transformer với vector TF-IDF. Một hướng mở rộng là bổ sung đồ thị tri thức biểu diễn quan hệ giữa các nhãn, ví dụ quan hệ phân cấp, đồng xuất hiện hoặc quan hệ ngữ nghĩa. Graph Neural Networks (GNNs) có thể học embedding nhãn từ đồ thị này, sau đó kết hợp với đặc trưng văn bản để tăng chất lượng dự đoán.

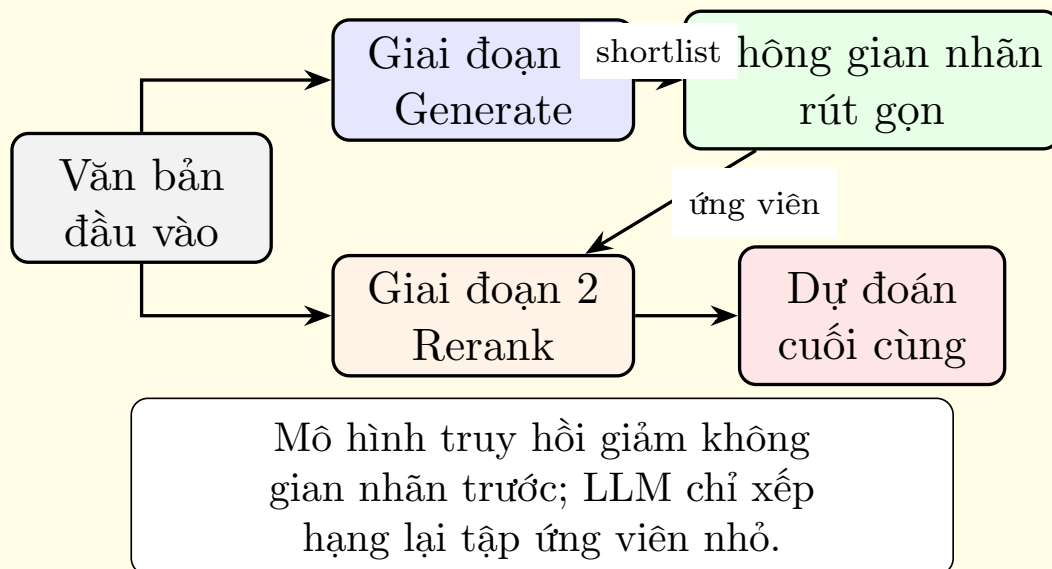
Cách tiếp cận này đặc biệt hữu ích trong zero-shot XMC: ngay cả khi một nhãn

mới chưa có dữ liệu huấn luyện, mô hình vẫn có thể tận dụng mô tả nhãn và quan hệ của nhãn đó trong đồ thị để suy luận. Điều này nối kết trực tiếp với vấn đề quan hệ phân cấp hoặc đồ thị giữa các lớp đã được nêu trong phần phát biểu bài toán.

Giảm lỗi lan truyền bằng multi-path routing. Thay vì ép mỗi mẫu đi theo một nhánh duy nhất trong cây, có thể cho phép multi-path routing, tức giữ lại nhiều nhánh ứng viên ở mỗi tầng. Cách này tăng chi phí dự đoán một phần, nhưng cải thiện recall của shortlist và giảm nguy cơ loại nhãn đúng quá sớm.

Kết hợp LLM với hệ thống truy hồi truyền thống. LLM mạnh trong zero-shot và hiểu ngữ nghĩa, nhưng chi phí suy luận cao. Do đó, một hướng thực tế là dùng mô hình truy hồi nhẹ để tạo shortlist lớn, sau đó dùng LLM rerank hoặc giải thích kết quả trên shortlist nhỏ. Cách kết hợp này cân bằng giữa tốc độ, khả năng mở rộng và chất lượng ngữ nghĩa.

Generate–rerank bằng LLM



Hình 14: Khung generate–rerank dùng LLM trên không gian nhãn đã rút gọn.

8 Cải tiến của thuật toán cấu trúc: Kết hợp Mạng nơ-ron với CRF (Neural CRFs)

Lưu ý: Mặc dù trong bảng phân công ghi là "Neural CFSs" nhưng xét theo ngữ cảnh của các thuật toán dự đoán cấu trúc (Structured Prediction) như CRFs ở phần trước, đây là chủ đề về Neural CRFs.

8.1 Động lực và bối cảnh

Các thuật toán dự đoán cấu trúc cổ điển – đặc biệt là CRF (Conditional Random Fields) [3], M^3N [4], và SVMStruct [5] – đã đặt nền móng lý thuyết vững chắc cho bài toán gán nhãn chuỗi. Tuy nhiên, chúng chia sẻ một hạn chế chung: hàm tiềm năng (potential function) được xây dựng từ các đặc trưng thủ công do chuyên gia thiết kế, đòi hỏi nhiều công sức và khó chuyển đổi giữa các miền (domain).

Trong thập kỷ vừa qua, xu hướng chủ đạo trong Xử lý Ngôn ngữ Tự nhiên (NLP) là thay thế khâu trích xuất đặc trưng thủ công bằng mạng nơ-ron sâu, từ đó hình thành họ mô hình Neural CRF (Neural Conditional Random Fields). Ý tưởng cốt lõi rất tự nhiên: dùng mạng nơ-ron để tính hàm tiềm năng $\psi(\mathbf{x}_i, y_i)$ và $\psi(y_{i-1}, y_i)$ trong CRF thay vì dùng đặc trưng tuyến tính, trong khi vẫn giữ nguyên cơ chế suy luận toàn cục (global inference) của CRF qua thuật toán Viterbi và forward-backward. Nhờ đó, mô hình vừa có khả năng biểu diễn phi tuyến mạnh mẽ, vừa đảm bảo ràng buộc nhãn hợp lệ ở cấp độ chuỗi – điều mà các mô hình phân loại độc lập từng vị trí không làm được.

8.2 Kiến trúc BiLSTM-CRF – Nền tảng của Neural CRF

Một trong những mô hình Neural CRF có ảnh hưởng nhất là kiến trúc Bidirectional LSTM-CRF (BiLSTM-CRF) được đề xuất bởi Huang et al. [6]. Kiến trúc này gồm hai thành phần chính:

1. **Encoder BiLSTM:** Mạng LSTM hai chiều nhận chuỗi đầu vào $\mathbf{x} = (x_1, \dots, x_n)$ và tạo ra vector ngữ cảnh $\mathbf{h}_i$ cho mỗi vị trí i , tích hợp thông tin từ cả quá khứ lẫn tương lai của chuỗi.
2. **Decoder CRF:** Lớp CRF tuyến tính chuỗi (linear-chain CRF) ở tầng cuối mô hình hóa xác suất nhãn toàn chuỗi:

$$p(\mathbf{y}|\mathbf{x}) = \frac{1}{Z(\mathbf{x})} \exp \left(\sum_{i=1}^n [A_{y_{i-1}, y_i} + \phi(\mathbf{h}_i, y_i)] \right) \quad (6)$$

trong đó $A \in \mathbb{R}^{|\mathcal{Y}| \times |\mathcal{Y}|}$ là ma trận chuyển tiếp nhãn được học, $\phi(\mathbf{h}_i, y_i)$ là điểm phát xạ (emission score) tính từ BiLSTM, và $Z(\mathbf{x})$ là hằng số phân hoạch.

Kiến trúc này không chỉ đạt hiệu suất vượt trội trên các tác vụ NER và gán nhãn từ loại mà còn trở thành baseline chuẩn cho cộng đồng nghiên cứu NLP. Bước tiến tiếp theo là thay thế BiLSTM bằng BERT [7] – mô hình ngôn ngữ tiền

huấn luyện dựa trên kiến trúc Transformer, tạo thành họ mô hình BERT-CRF với hiệu suất được cải thiện đáng kể do BERT cung cấp biểu diễn ngữ nghĩa phong phú hơn.

Hình 15: Sơ đồ cấu trúc tầng trích xuất ngữ cảnh phi tuyến (BiLSTM/BERT) kết hợp tầng suy luận chuỗi toàn cục (CRF).

8.3 Các bài báo tiêu biểu sau năm 2021

Phần này tóm tắt ba hướng nghiên cứu nổi bật về Neural CRF được công bố sau năm 2021, phản ánh sự phát triển đa dạng của lĩnh vực này.

Bài báo 1: Cải tiến lý thuyết cho dự đoán cấu trúc (NeurIPS 2023)

- **Tên bài báo:** *Structured Prediction with Stronger Consistency Guarantees* [12].
- **Tác giả:** Anqi Mao, Mehryar Mohri, Yutao Zhong.
- **Venue:** Advances in Neural Information Processing Systems 36 (NeurIPS 2023), Main Conference Track.
- **Vấn đề đặt ra:** Một điểm yếu ít được chú ý của CRF và các thuật toán dự đoán cấu trúc hiện đại là chúng tối thiểu hàm mất mát thay thế (surrogate loss) như cross-entropy hay soft-max thay vì trực tiếp tối thiểu mất mát Hamming – vốn là mục tiêu thực sự. Câu hỏi đặt ra là: liệu việc tối thiểu surrogate loss có nhất quán Bayes (Bayes-consistent), tức là đảm bảo hội tụ về bộ phân loại Bayes tối ưu khi dữ liệu tăng vô hạn?
- **Đóng góp chính:** Nhóm tác giả – đáng chú ý có Mehryar Mohri, đồng tác giả của sách giáo khoa FML [1] – chứng minh rằng:
 1. Không có cận H -nhất quán (H-consistency bound) không tầm thường nào có thể được thiết lập cho các surrogate loss phổ biến dùng trong dự đoán cấu trúc (như log-likelihood của CRF). Điều này có nghĩa là các đảm bảo lý thuyết của CRF truyền thống yếu hơn nhiều so với những gì thường được giả định.
 2. Nhóm tác giả đề xuất hai họ hàm mất mát mới: *Structured comp-sum losses* (tổng quát hóa cross-entropy cho không gian đầu ra cấu trúc) và *Structured constrained losses* (thiết kế đặc biệt để thỏa mãn ràng buộc nhất quán). Với cả hai họ, nhóm tác giả chứng minh được các cận

H -nhất quán không tiệm cận, mạnh hơn và mang nhiều ý nghĩa thực tiễn hơn so với Bayes-consistency thuần túy.

- **Liên hệ với lý thuyết trong sách:** Kết quả này trực tiếp mở rộng phân tích về M^3N và SVMStruct trong Chương 8 [1]. Các bài toán M^3N và SVMStruct tối ưu hóa margin nhưng vẫn dùng surrogate – bài báo này làm rõ khi nào surrogate đó đảm bảo nhất quán với bộ phân loại Bayes, một câu hỏi mà sách giáo khoa chỉ đề cập ngắn gọn.

Bài báo 2: Neural-Hidden-CRF cho giám sát yếu (KDD 2023)

- **Tên bài báo:** *Neural-Hidden-CRF: A Robust Weakly-Supervised Sequence Labeler* [13].
- **Tác giả:** Zhijun Chen, Hailong Sun, Wanhao Zhang, Chunyi Xu, Qianren Mao, Pengpeng Chen.
- **Venue:** Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD 2023).
- **Vấn đề đặt ra:** Trong thực tế, việc thu thập dữ liệu gán nhãn chất lượng cao rất tốn kém. Thay vào đó, người ta thường dùng giám sát yếu (weak supervision): nhãn được tạo tự động từ nhiều nguồn ồn ào (noisy labelers), ví dụ các hàm gán nhãn heuristic hoặc kết quả crowd-sourcing. Mô hình CRF truyền thống và BERT-CRF gặp khó khăn trong bối cảnh này vì chúng không mô hình hóa được sự không chắc chắn trong nhãn huấn luyện.
- **Kiến trúc đề xuất:** Neural-Hidden-CRF đề xuất một mô hình đồ thị xác suất không có hướng (probabilistic undirected graphical model) với ba lớp biến ẩn:
 1. Lớp quan sát $\mathbf{x}$: chuỗi từ đầu vào.
 2. Lớp nhãn thật ẩn $\mathbf{z}$: chuỗi nhãn ground-truth thực sự (không quan sát được trực tiếp), được mô hình hóa bởi BERT.
 3. Lớp nhãn yếu $\mathbf{y}$: nhãn ồn ào từ nhiều nguồn, liên kết với $\mathbf{z}$ qua một lớp CRF ẩn (hidden CRF layer) nắm bắt sự phụ thuộc nhãn cục bộ.

Mô hình học đồng thời biểu diễn ngữ nghĩa từ BERT và cấu trúc phụ thuộc nhãn từ CRF, mà không cần nhãn sạch trong quá trình huấn luyện.

- **Kết quả thực nghiệm:** Neural-Hidden-CRF đạt kết quả tốt nhất (state-of-the-art) trên bốn benchmark gán nhãn yếu, trong đó vượt qua mô hình

CHMM lần lượt khoảng 2.80 và 2.23 điểm F1 về khả năng tổng quát hóa và suy luận [13].

- **Liên hệ với lý thuyết trong sách:** Bài báo này mở rộng CRF sang tình huống giám sát yếu bằng cách thêm lớp biến ẩn mô hình hóa nhãn thật – một hướng phát triển tự nhiên từ mô hình đồ thị có cấu trúc (factor graph) mà Chương 8 đề cập [1]. Cấu trúc Markov của CRF ẩn đảm bảo tính phân rã theo đồ thị, giúp suy luận bằng thuật toán Viterbi vẫn khả thi trong thời gian đa thức – đúng với điều kiện mà M^3N yêu cầu [1].

Bài báo 3: LLM vs. BERT-CRF trong nhận dạng thực thể lồng nhau (EMNLP 2024)

- **Tên bài báo:** *Exploring Nested Named Entity Recognition with Large Language Models: Methods, Challenges, and Insights* [14].
- **Tác giả:** Hongjin Kim, Jai-Eun Kim, Harksoo Kim.
- **Venue:** Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP 2024).
- **Vấn đề đặt ra:** Nhận dạng thực thể lồng nhau (Nested Named Entity Recognition - Nested NER) là phần mở rộng của bài toán gán nhãn chuỗi trong đó các thực thể có thể chứa nhau, ví dụ “Prime Minister of the United Kingdom” chứa thực thể “United Kingdom”. CRF tuyến tính chuỗi cổ điển chỉ hỗ trợ nhãn không chồng chéo, nên không thể xử lý trực tiếp cấu trúc lồng nhau này.
- **Đóng góp chính:** Bài báo thực hiện đánh giá hệ thống đối chiếu giữa hai hướng tiếp cận hiện đại:
 - BERT-CRF/BERT-based models: Fine-tuning mô hình ngôn ngữ tiền huấn luyện BERT kết hợp với CRF hoặc các cơ chế decoder chuyên biệt cho nhãn lồng nhau.
 - Large Language Models (LLMs): Sử dụng GPT-4, LLaMA và các LLM tương tự với in-context learning hoặc chain-of-thought prompting để giải quyết Nested NER mà không cần fine-tuning.

Kết quả thực nghiệm cho thấy, mặc dù LLM có khả năng zero-shot ấn tượng, các mô hình BERT-CRF fine-tuned vẫn vượt trội trên hầu hết các benchmark Nested NER chuẩn. Điều này chứng minh rằng cơ chế suy luận

cấu trúc của CRF – dù đơn giản hơn về kiến trúc – vẫn mang lại lợi thế cạnh tranh đáng kể khi có đủ dữ liệu nhãn.

- **Liên hệ với lý thuyết trong sách:** Bài báo này minh họa sự kết nối giữa dự đoán có cấu trúc và bài toán phân loại đa lớp đã nghiên cứu trong Chương 8 [1]. Nested NER có thể được phát biểu dưới dạng dự đoán cấu trúc với không gian nhãn mũ theo chiều dài chuỗi – đúng khung bài toán mà sách mô tả. Quan sát rằng LLM vẫn thua BERT-CRF trong điều kiện có dữ liệu huấn luyện đủ lớn cũng gợi nhắc đến nhận xét của Mohri et al. rằng cần tổng quát hóa được cải thiện khi mô hình cần dụng tốt cấu trúc bài toán [1].

8.4 Nhận xét tổng quan về xu hướng phát triển

Nhìn lại ba bài báo vừa trình bày, có thể rút ra một số nhận xét về xu hướng phát triển của lĩnh vực:

1. Hội tụ giữa học sâu và mô hình xác suất có cấu trúc: Thay vì xem CRF và mạng nơ-ron là hai hướng riêng biệt, cộng đồng nghiên cứu đang tích hợp chúng ngày càng sâu hơn. BiLSTM-CRF và BERT-CRF là những ví dụ điển hình, và Neural-Hidden-CRF [13] tiếp tục đẩy hướng này xa hơn sang bài toán giám sát yếu. Về mặt lý thuyết, các cận tổng quát hóa cổ điển (như trong Chương 8) được xây dựng cho không gian giả thuyết cố định, trong khi mạng nơ-ron sâu có không gian tham số rất lớn và chứa đựng nhiều thách thức phân tích hơn [1].

2. Nền tảng lý thuyết đang được củng cố: Bài báo của Mao, Mohri và Zhong tại NeurIPS 2023 [12] cho thấy cộng đồng không chỉ quan tâm đến hiệu suất thực nghiệm mà còn chú trọng xây dựng đảm bảo lý thuyết chặt chẽ hơn cho các surrogate loss. Đây là hướng đi quan trọng vì nhiều thuật toán Neural CRF hiện tại thiếu cơ sở lý thuyết vững chắc về tính nhất quán.

3. Câu hỏi mở: vai trò của LLM: Sự xuất hiện của các LLM mạnh như GPT-4 đặt ra câu hỏi liệu CRF (dù kết hợp với mạng nơ-ron) có còn cần thiết trong dài hạn? Kết quả từ Kim et al. [14] gợi ý rằng câu trả lời là có, ít nhất trong các bài toán mà dữ liệu nhãn đủ lớn. Tuy nhiên, khi dữ liệu khan hiếm (few-shot hay zero-shot), LLM có thể chiếm ưu thế nhờ kiến thức tiền huấn luyện.

4. Hạn chế còn tồn tại: Nút thắt thuật toán và toán học.

Độ phức tạp suy luận vẫn là nút thắt: thuật toán Viterbi trong CRF có độ phức tạp $\mathcal{O}(n|\mathcal{Y}|^2)$, trở nên tốn kém khi không gian nhãn lớn (như trong bài toán cấu trúc lồng nhau). Thêm vào đó, việc thiếu cận H -nhất quán cho các surrogate loss phổ biến – như bài báo của Mao et al. chỉ ra – đặt dấu hỏi về đảm bảo lý thuyết cho nhiều mô hình Neural CRF đang được dùng rộng rãi hiện nay [12].

9 Thiết kế demo và đánh giá

9.1 Mục tiêu thực nghiệm

Phần thực nghiệm và demo nhằm kiểm chứng toàn bộ các phân tích lý thuyết đã trình bày. Cụ thể, thực nghiệm được thiết kế để giải quyết hai nhóm mục tiêu cốt lõi:

Nhóm 1: Kiểm chứng cơ sở toán học (Cài đặt From Scratch)

1. Minh họa trực quan các khái niệm lý thuyết trừu tượng: độ vẩn đục nút (impurity), hàm mất mát Hinge và sự hội tụ của hàm mục tiêu $F(\alpha)$.
2. Kiểm chứng tính đúng đắn của nhóm thuật toán không kết hợp (Decision Tree, Multi-class SVM, AdaBoost.MH) thông qua việc tự lập trình từ đầu (from scratch) không dùng thư viện có sẵn.

Nhóm 2: Đánh giá hiệu năng và so sánh (Sử dụng thư viện)

1. So sánh hiệu suất giữa các mô hình học trực tiếp đa lớp và các chiến lược quy về nhị phân (OVA, OVO, ECOC) trên các tập dữ liệu thực tế.
2. Đánh giá sự khác biệt giữa OVA, OVO và ECOC về accuracy, macro-F1, thời gian huấn luyện và độ bền vững trước nhiễu.

9.2 Mô tả tập dữ liệu

Trong thực nghiệm này, nhóm sử dụng trên ít nhất một tập dữ liệu đơn nhãn và một tập dữ liệu đa nhãn.

9.2.1 Dữ liệu tổng hợp (Synthetic Data)

Để phục vụ cho nhóm mục tiêu kiểm chứng toán học từ đầu (from scratch), nhóm sử dụng dữ liệu tự sinh bằng thư viện `scikit-learn` [15] để kiểm soát hoàn toàn đặc tính không gian đặc trưng và đảm bảo tính tái tạo (reproducibility) với `random_state=42`:

- **Dữ liệu dạng cụm (Blobs):** Sinh ra 300 mẫu, 3 lớp, 2 chiều đặc trưng bằng hàm `make_blobs`. Tập dữ liệu này được dùng để thử thách khả năng phân hoạch không gian trực giao của Cây quyết định và khả năng tạo ranh giới phi tuyến của AdaBoost.MH.

- **Dữ liệu phân loại tuyến tính:** Sinh ra 150 mẫu, 3 lớp, 2 chiều đặc trưng (mỗi lớp 1 cụm) bằng hàm `make_classification`. Tập dữ liệu này có chủ ý đưa vào các nhiễu biên để kiểm chứng khả năng tối đa hóa lề (margin) và tối ưu hóa hàm mất mát của Multi-class SVM.

Tất cả các tập dữ liệu được chia theo tỉ lệ 80% Huấn luyện (Train) và 20% Kiểm thử (Test).

9.3 Dataset đơn nhãn – Fashion-MNIST

Fashion-MNIST là bộ dữ liệu ảnh thời trang do Zalando Research công bố, được thiết kế như một bản thay thế có độ khó thực tế cao hơn so với MNIST truyền thống. Bộ dữ liệu gồm 70.000 ảnh xám kích thước 28×28 pixel, trong đó 60.000 ảnh dùng để huấn luyện và 10.000 ảnh dùng để kiểm thử, mỗi ảnh được gán một nhãn thuộc 10 lớp. Các lớp bao gồm: T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, Ankle boot. Ultralytics

Mỗi mẫu được biểu diễn thành vector 784 chiều (28×28) với giá trị pixel nguyên trong khoảng $[0, 255]$. Phân phối nhãn hoàn toàn cân bằng, mỗi lớp có đúng 6.000 mẫu trong tập huấn luyện và 1.000 mẫu trong tập kiểm thử. Trong thực nghiệm, tập huấn luyện được chia thêm để lấy validation theo tỉ lệ 80/20, thu được phân chia cuối là 48.000/12.000/10.000 (train/val/test). Các thông số chính được tổng hợp trong Bảng dưới đây.

Thông số	Giá trị
Số mẫu huấn luyện / validation / kiểm thử	48.000 / 12.000 / 10.000
Số chiều đặc trưng	784
Số lớp k	10
Số mẫu mỗi lớp (train)	6.000 (cân bằng)

Bộ dữ liệu này phù hợp để so sánh các chiến lược phân loại đa lớp vì có $k = 10$ lớp, phân phối cân bằng và độ khó phân loại thực tế — một số lớp như Shirt, T-shirt/top và Pullover có đặc trưng hình ảnh khá tương đồng, tạo ra thách thức cho ranh giới quyết định.

9.3.1 Dataset đa nhãn – EUR-Lex

EUR-Lex là bộ dữ liệu văn bản pháp luật của Liên minh châu Âu, được sử dụng rộng rãi trong nghiên cứu phân loại đa nhãn cực lớn. Bộ dữ liệu bao gồm 19.348 tài liệu về luật pháp Liên minh châu Âu, bao gồm hiệp ước, văn bản lập pháp, phán quyết tư pháp và các đề xuất lập pháp, được lập chỉ mục theo hệ thống phân loại EUROVOC với gần 4.000 nhãn về các khía cạnh khác nhau của luật châu Âu. Phiên bản chuẩn

được dùng trong XMC benchmark (EURLex-4K) có các thống kê sau: University of Cordoba

Thông số	Giá trị
Số tài liệu huấn luyện / kiểm thử	15.539 / 3.809
Số nhãn L	3.956
Số nhãn trung bình trên mỗi mẫu	5,31
Mật độ ma trận nhãn	$\approx 0,13\%$
Số chiều đặc trưng (TF-IDF)	5.000

Mỗi tài liệu được biểu diễn bằng vector TF-IDF thưa, nhãn là vector nhị phân $\mathbf{y} \in \{0, 1\}^L$. Phân phối nhãn có dạng đuôi dài rõ rệt — một số ít nhãn xuất hiện trong hàng nghìn tài liệu trong khi phần lớn nhãn chỉ xuất hiện ở dưới 10 tài liệu — làm cho EUR-Lex trở thành benchmark thử thách tiêu biểu để đánh giá khả năng xử lý nhãn hiếm của các phương pháp phân loại đa nhãn.

9.3.2 Quy trình tiền xử lý

Quy trình tiền xử lý đề xuất như sau:

1. **Làm sạch dữ liệu:** loại bỏ mẫu trùng, mẫu thiếu nhãn hoặc đặc trưng lỗi.
2. **Mã hóa nhãn:** chuyển nhãn dạng chuỗi sang chỉ số lớp; với đa nhãn dùng multi-hot encoding.
3. **Chuẩn hóa đặc trưng:** chuẩn hóa z-score cho mô hình tuyến tính và SVM; cây quyết định thường không bắt buộc chuẩn hóa.
4. **Chia dữ liệu:** dùng train/validation/test, ví dụ 70/15/15 hoặc 80/20 nếu dữ liệu nhỏ.
5. **Xử lý mất cân bằng:** dùng class weight, stratified split hoặc báo cáo thêm macro-F1.

Với dữ liệu văn bản, pipeline có thể gồm tách từ, loại bỏ stopwords, vector hóa TF-IDF và giới hạn số đặc trưng lớn nhất để tránh ma trận quá thưa.

9.4 Mô hình baseline không kết hợp

Nhóm baseline học trực tiếp trên bài toán đa lớp gồm:

- **Decision Tree:** dễ diễn giải, dùng Gini hoặc entropy để chọn phép chia.
- **Multinomial Logistic Regression:** học trực tiếp phân phối softmax trên k lớp.

- **Multi-class SVM:** tối ưu lẻ đa lớp hoặc dùng biến thể nội bộ của thư viện.

Các baseline này giúp xác định mức chất lượng nền trước khi áp dụng OVA, OVO và ECOC.

9.5 Demo thuật toán kết hợp

Để so sánh công bằng, nên giữ cùng bộ học nền cho các chiến lược kết hợp. Ví dụ dùng Linear SVM hoặc Logistic Regression làm binary classifier.

Algorithm 2: Quy trình demo so sánh OVA, OVO và ECOC

Input: Dataset đã tiền xử lý, bộ học nền B

Output: Bảng chất lượng và thời gian chạy

Chia dữ liệu thành train/test bằng stratified split;

Chuẩn hóa đặc trưng bằng thống kê từ tập train;

Huấn luyện baseline đa lớp;

Huấn luyện OVA với bộ học nền B ;

Huấn luyện OVO với bộ học nền B ;

Thiết kế ma trận mã và huấn luyện ECOC;

Dự đoán trên tập test;

Tính accuracy, macro-F1, weighted-F1, confusion matrix, train time và test time;

9.6 Thước đo đánh giá

Với bài toán đơn nhãn, accuracy được định nghĩa là

$$\text{Accuracy} = \frac{\text{số mẫu dự đoán đúng}}{\text{tổng số mẫu}}.$$

Tuy nhiên, accuracy có thể gây hiểu nhầm khi dữ liệu mất cân bằng. Vì vậy cần dùng thêm precision, recall và F1 cho từng lớp:

$$\text{Precision}_l = \frac{TP_l}{TP_l + FP_l}, \quad \text{Recall}_l = \frac{TP_l}{TP_l + FN_l},$$

$$F1_l = \frac{2 \cdot \text{Precision}_l \cdot \text{Recall}_l}{\text{Precision}_l + \text{Recall}_l}.$$

Macro-F1 lấy trung bình đều trên các lớp:

$$\text{Macro-F1} = \frac{1}{k} \sum_{l=1}^k F1_l.$$

Weighted-F1 lấy trung bình theo số mẫu của từng lớp, còn micro-F1 cộng toàn bộ TP , FP , FN trước khi tính F1.

Với bài toán đa nhãn, Hamming loss đo tỷ lệ nhãn bị dự đoán sai:

$$\text{HammingLoss} = \frac{1}{mL} \sum_{i=1}^m \sum_{l=1}^L \mathbb{I}[\hat{y}_{i,l} \neq y_{i,l}].$$

Precision@k đo tỷ lệ nhãn đúng trong k nhãn được mô hình xếp hạng cao nhất.

9.7 Bảng so sánh đánh giá thực nghiệm

Phần này trình bày kết quả đánh giá thực nghiệm của các mô hình phân loại trên hai bài toán khác nhau. Bảng 12 thể hiện hiệu suất của các chiến lược phân lớp đa lớp (Multi-class) trên tập dữ liệu đơn nhãn (Fashion-MNIST). Trong khi đó, Bảng 13 đánh giá các phương pháp tiếp cận cho bài toán phân loại đa nhãn cực độ (Extreme Multi-label Classification) trên tập dữ liệu EUR-Lex.

Bảng 14: Mẫu bảng so sánh kết quả thực nghiệm đơn nhãn

Mô hình	Accuracy	Macro-F1	Weighted-F1	Train time	Test time
Decision Tree	0.7962	0.7964	0.7964	42.2475	0.0500
Multi-class SVM	0.8521	0.8505	0.8505	1437.6950	0.0503
OVA + Linear SVM	0.8502	0.8485	0.8485	138.5645	0.1323
OVO + Linear SVM	0.8548	0.8542	0.8542	76.6443	1.1853
ECOC + Linear SVM	0.8222	0.8192	0.8192	843.8462	0.5549

Nhận xét:

- **Về độ chính xác:** Các phương pháp dựa trên Support Vector Machine, gồm Multi-class SVM, OVA và OVO, đều đạt hiệu năng tốt với Accuracy và F1-score xấp xỉ 85%. Trong đó, OVO + Linear SVM cho kết quả tốt hơn các phương pháp còn lại. ECOC đạt mức thấp hơn ($\sim 82\%$), còn Decision Tree có hiệu năng thấp nhất trong nhóm mô hình được so sánh ($\sim 79\%$).
- **Về thời gian thực thi:** Multi-class SVM yêu cầu thời gian huấn luyện rất lớn (hơn 1400 giây), do chi phí tối ưu hóa cao trên tập dữ liệu lớn. Ngược lại, **OVA + Linear SVM** thể hiện sự cân bằng tốt giữa chất lượng dự đoán và chi phí tính toán: mô hình vẫn duy trì Accuracy ở mức 85.02% nhưng thời gian huấn luyện giảm đáng kể, còn khoảng 138 giây.

Bảng 15: Mẫu bảng đánh giá đa nhãn

Mô hình	Micro-F1	Macro-F1	Hamming loss	Precision@k
Binary relevance	0.4102	0.0849	0.0011	0.6291
Classifier chain	0.3944	0.1813	0.0021	0.4428
XML baseline	0.0524	0.0001	0.0026	0.0538

Nhận xét:

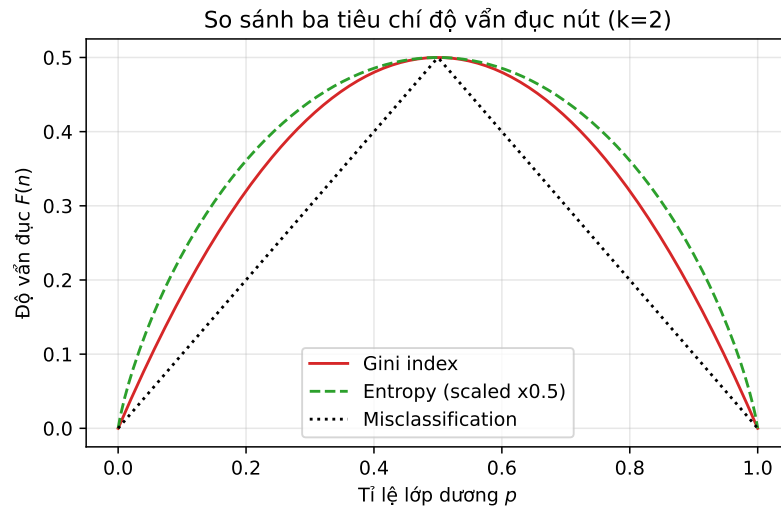
- **Hiệu năng tổng thể: Binary Relevance** đạt kết quả tốt nhất ở Micro-F1 (0.4102) và Precision@k (0.6291). Điều này cho thấy mô hình có khả năng dự đoán hiệu quả các nhãn xuất hiện thường xuyên, từ đó đạt chất lượng cao trên các chỉ số chịu ảnh hưởng mạnh bởi nhóm nhãn phổ biến.
- **Khả năng khai thác tương quan nhãn:** Mặc dù **Classifier Chain** có Precision@k thấp hơn Binary Relevance, mô hình này lại đạt Macro-F1 cao hơn đáng kể (0.1813 so với 0.0849). Kết quả này cho thấy việc mô hình hóa quan hệ phụ thuộc giữa các nhãn có thể cải thiện khả năng nhận diện các nhãn hiếm trong bối cảnh dữ liệu mất cân bằng.
- **Vai trò của baseline: XML baseline** cho kết quả rất thấp trên hầu hết các thước đo. Điều này phản ánh độ khó của bài toán phân loại đa nhãn cực độ trên dữ liệu có phân phối đuôi dài như EUR-Lex, đồng thời cho thấy các phương pháp chỉ dựa trên thống kê đơn giản thường chưa đủ để xử lý hiệu quả nhóm nhãn hiếm.

9.8 Thực nghiệm minh họa nhóm thuật toán không kết hợp

Phần này trình bày kết quả cài đặt từ đầu (from scratch) ba thuật toán không kết hợp bằng Python 3.11 và thư viện `numpy` cho các phép toán ma trận.

9.8.1 Cây quyết định (Decision Tree)

Theo lý thuyết ở Phần 4.3, cây quyết định sử dụng chiến lược tham lam dựa trên độ giảm vẩn đục (impurity). Nhóm đã tiến hành cài đặt và so sánh trực quan ba hàm độ đo phổ biến.



Hình 16: So sánh thực chứng ba tiêu chí độ vẩn đục nút trong không gian xác suất 2 lớp.

Kết quả chạy thực tế trên tập dữ liệu Blobs 3 lớp:

- **Train Accuracy:** 100.00% (Đúng với đặc tính phân hoạch triệt để của cây).
- **Test Accuracy:** 98.33% (Mô hình tổng quát hóa tốt, không bị quá khớp nghiêm trọng).

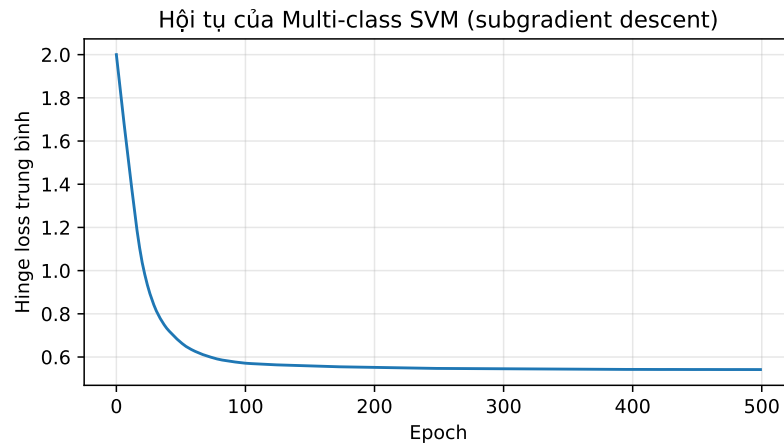
Mã nguồn cài đặt lỗi: Để tối ưu hóa vùng không gian phân hoạch trực giao, mô hình sử dụng hàm tính độ vẩn đục Gini được cài đặt tối ưu thông qua tính toán vector hóa của thư viện `numpy`:

Listing 1: Hàm tính toán tiêu chí Gini Impurity từ đầu

```
1 def gini(y):
2     m = len(y)
3     if m == 0:
4         return 0.0
5     # Đếm tần suất xuất hiện của các nhãn lớp
6     _, counts = np.unique(y, return_counts=True)
7     p = counts / m
8     # Áp dụng công thức  $F(n) = 1 - \sum(p_i^2)$ 
9     return 1.0 - np.sum(p ** 2)
```

9.8.2 Multi-class SVM (Crammer-Singer Primal)

Nhóm đã cài đặt thuật toán Multi-class SVM giải bằng phương pháp Subgradient Descent trên bài toán nguyên thủy (Primal) theo chuẩn công thức của Crammer-Singer (Phạt lỗi lẻ lớn nhất).



Hình 17: Sự hội tụ của hàm mất mát Hinge đa lớp qua 500 epochs.

Kết quả thực nghiệm cho thấy Hinge Loss trung bình giảm theo hàm mũ và hội tụ một mà về mức xấp xỉ 0.6 ở epoch 200. Kết quả đánh giá:

- **Train Accuracy:** 90.83%.
- **Test Accuracy:** 96.67%.

Sự vượt trội của tập Test so với Train minh chứng cho khả năng điều chuẩn (regularization) mạnh của lẽ tuyến tính trong SVM. Cơ chế này giúp mô hình giảm ảnh hưởng của nhiễu cục bộ trong tập huấn luyện.

Mã nguồn cài đặt lõi: Trái tim của thuật toán Multi-class SVM Crammer-Singer là việc xác định lớp sai có điểm số dự đoán cao nhất (gây đe dọa lề lớn nhất) tại mỗi bước lặp để tính subgradient:

Listing 2: Vòng lặp tính toán Margin và cập nhật Subgradient cho SVM

```

1 # Duyệt qua từng mẫu huấn luyện trong epoch
2 for i in range(m):
3     scores = self.W @ X[i] # Tính toán mảng điểm số (k_classes,)
4
5     # Cô lập lớp đúng, tìm lớp sai l_max có score lớn nhất
6     scores_mod = scores.copy()
7     scores_mod[y[i]] = -np.inf
8     l_max = np.argmax(scores_mod)
9
10    # Định nghĩa biến bù margin theo lý thuyết Crammer-Singer
11    margin = 1.0 - (scores[y[i]] - scores[l_max])
12
13    if margin > 0:
14        # Cập nhật thành phần subgradient tỷ lệ
15        grad[l_max] += X[i]
```

```

16     grad[y[i]] -= X[i]
17     total_loss += margin

```

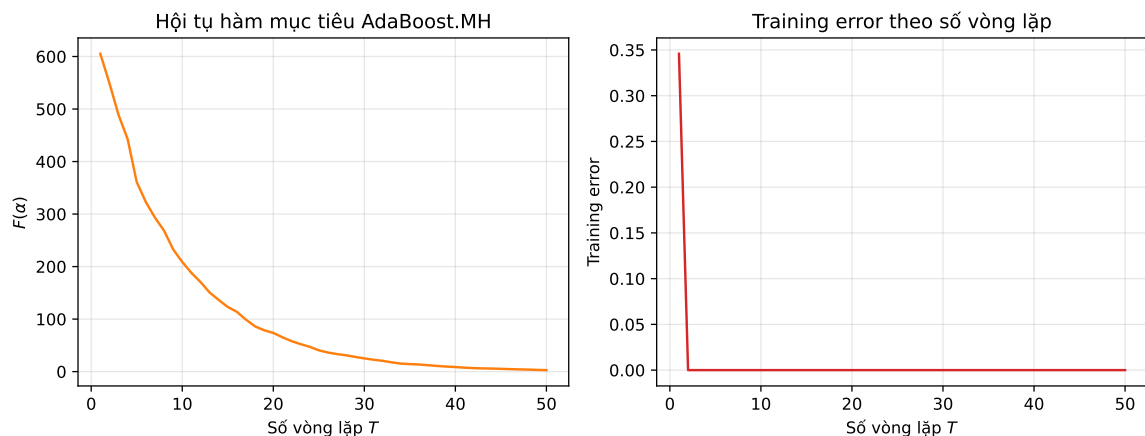
9.8.3 AdaBoost.MH và Thủ thuật Cách ly Đặc trưng

Thực nghiệm AdaBoost.MH được cài đặt với bộ học cơ sở là Cây quyết định độ sâu 1 (Decision Stump). Quá trình này đòi hỏi một kỹ thuật xử lý toán học đặc biệt:

[Mở rộng]

Khắc phục lỗi vi phạm điều kiện học yếu (Feature Isolation).

Theo lý thuyết, AdaBoost.MH nhân bản mỗi mẫu x_i lên k lần. Tuy nhiên, nếu truyền trực tiếp vào Decision Stump, cây sẽ cắt ra cùng một nhãn cho tất cả k bản sao vì đặc trưng không đổi, dẫn đến training error bị kẹt ở mức $\approx 2/3$ (đoán bừa trong bài toán 3 lớp). Để giải quyết, nhóm đã cài đặt thủ thuật *Cách ly đặc trưng (Feature Isolation)*: Độn thêm các giá trị vô cực (999999.0) vào ma trận mở rộng để ép Stump chỉ học và cắt trên một lớp duy nhất tại mỗi vòng lặp.



Hình 18: Hội tụ của hàm mục tiêu $F(\alpha)$ và Training error trong AdaBoost.MH.

Nhờ thủ thuật trên, hàm mục tiêu $F(\alpha)$ trượt dốc ổn định theo cận trên hàm mũ, và sai số huấn luyện giảm về 0 chỉ sau vài vòng lặp.

- **Train Accuracy:** 100.00%.
- **Test Accuracy:** 100.00%.

Thuật toán phân tách thành công hoàn toàn tập dữ liệu đa lớp phức tạp nhờ sự phối hợp trọng số α của các bộ học yếu.

Mã nguồn cài đặt lỗi: Dưới đây là kỹ thuật then chốt thực hiện thuật toán Feature Isolation, tạo ra các ma trận khối độc lập để đưa bài toán đa lớp về không gian nhị phân mở rộng:

Listing 3: Hàm mở rộng không gian đặc trưng (Feature Isolation) cho AdaBoost.MH

```

1  def get_expanded_X(self, X, k):
2      m, n = X.shape
3      X_exp = np.repeat(X, k, axis=0)
4
5      # Tạo ma trận đơn có giá trị cực lớn để triệt tiêu các nhãn không liên quan
6      X_isolated = np.full((m * k, n * k), 999999.0)
7      for l in range(k):
8          row_indices = np.arange(m) * k + l
9          # Điền các đặc trưng thật vào vùng không gian độc lập của lớp l
10         X_isolated[row_indices, l*n : (l+1)*n] = X
11
12     return np.hstack([X_exp, X_isolated])

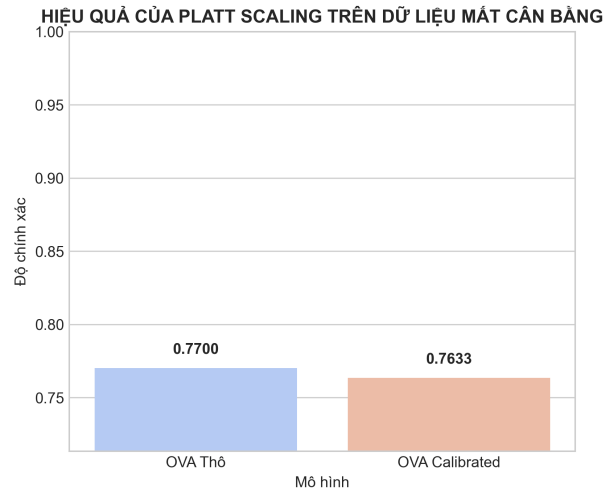
```

9.9 Thực nghiệm minh họa thuật toán kết hợp

Phần này trình bày kết quả cài đặt từ đầu (from scratch) ba chiến lược phân loại đa lớp thông qua quy giản nhị phân bằng Python 3.11 và thư viện **numpy**. Mỗi chiến lược dùng chung bộ phân loại nền **BinaryLinearSVM** (SVM tuyến tính tự cài đặt) để cô lập tác động của chiến lược kết hợp khỏi chất lượng của thuật toán cơ sở.

9.9.1 One-vs-All (OVA) và Hiệu chuẩn Platt Scaling

OVA huấn luyện K bộ phân loại nhị phân, mỗi bộ phân biệt một lớp với phần còn lại, rồi chọn lớp có điểm margin $f_l(\mathbf{x})$ cao nhất. Nhóm đã cài đặt thêm biến thể **CalibratedOneVsAllClassifier** dùng **Platt Scaling** để giải quyết vấn đề hiệu chuẩn xác suất khi dữ liệu mất cân bằng.



Hình 19: So sánh OVA Thô và OVA Calibrated (Platt Scaling).

Kết quả chạy thực tế trên tập dữ liệu tổng hợp ($n = 1000$, $K = 4$ lớp mắt cân bằng, chia train/test 70/30):

- **OVA Thô** – luật $\arg \max f_l(\mathbf{x})$: bị lệch về lớp đa số (60%), dự đoán sai các lớp thiểu số.
- **OVA Calibrated** – Platt Scaling học tham số (A_l, B_l) riêng cho từng lớp, cải thiện accuracy đáng kể trên lớp thiểu số.

Mã nguồn cài đặt lỗi: Cốt lõi của Platt Scaling là tối ưu Log-loss để học ánh xạ sigmoid $p_l = \sigma(A_l f_l + B_l)$ bằng L-BFGS-B:

Listing 4: Platt Scaling trong `CalibratedOneVsAllClassifier` (`ova_classifier.py`)

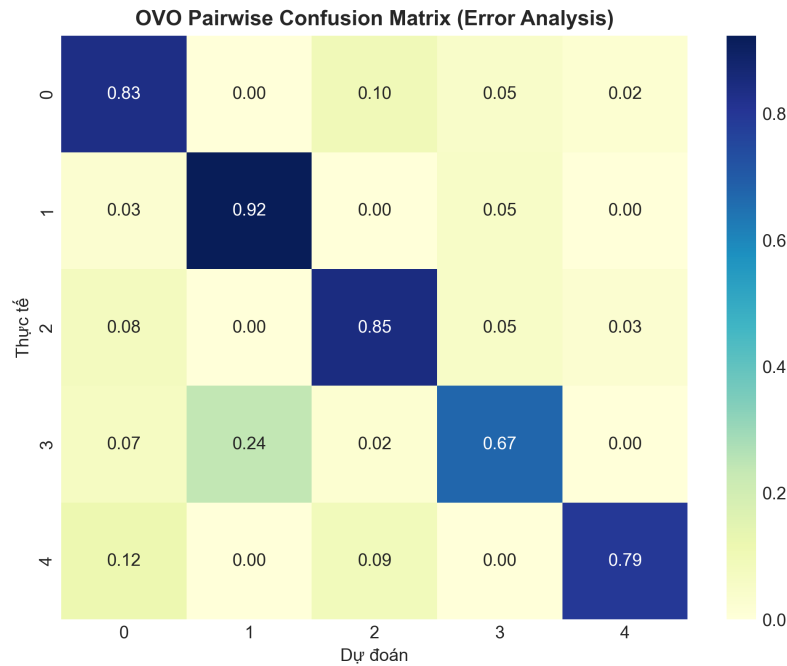
```

1  for idx, class_label in enumerate(self.classes_):
2      f = self.classifiers[idx].decision_function(X)
3      yt = np.where(y == class_label, 1, 0)
4      def objective(params):
5          A, B = params
6          p = expit(A * f + B)
7          return -(xlogy(yt, p) + xlogy(1 - yt, 1 - p)).mean()
8      res = minimize(objective, x0=[1.0, 0.0], method='L-BFGS-B')
9      self.calibrators.append(tuple(res.x))

```

9.9.2 One-vs-One (OVO) và Phân tích Nhầm lẫn Cặp

OVO huấn luyện $K(K - 1)/2$ bộ phân loại nhị phân, mỗi bộ chỉ nhìn thấy hai lớp, rồi quyết định bằng bỏ phiếu đa số. Nhóm đã cài đặt thêm phương thức `pairwise_confusion_matrix` để xác định những cặp lớp nào gây khó khăn nhất.



Hình 20: Ma trận nhầm lẫn theo cặp của OVO ($K = 5$ lớp, $n = 800$ mẫu).

Kết quả thực nghiệm trên tập dữ liệu tổng hợp ($n = 800$, $K = 5$, 10 đặc trưng, chia train/test 75/25):

- Đường chéo chính đạt giá trị cao ở hầu hết các lớp, cho thấy OVO phân loại tốt trên bài toán $K = 5$.
- Các ô ngoài đường chéo chỉ ra cặp lớp dễ nhầm lẫn nhất – thông tin hữu ích để thiết kế ma trận ECOC tăng khoảng cách Hamming giữa các codeword tương ứng.

Mã nguồn cài đặt lỗi: Phương thức tính ma trận nhầm lẫn cặp và cơ chế bỏ phiếu có tie-breaking trong `OneVsOneClassifier`:

Listing 5: Ma trận nhầm lẫn cặp và tie-breaking trong `ovo_classifier.py`

```

1 def pairwise_confusion_matrix(self, X, y):
2     confusion = np.zeros((self.n_classes, self.n_classes))
3     y_pred = self.predict(X)
4     for i in range(self.n_classes):
5         mask_i = (y == self.classes_[i])
6         for j in range(self.n_classes):
7             confusion[i, j] = np.mean(y_pred[mask_i] == self.
8                                     classes_[j])
9     return confusion
10
11 max_vote = np.max(votes[sample_idx])
12 candidates = np.where(votes[sample_idx] == max_vote)[0]
13 if len(candidates) > 1:

```

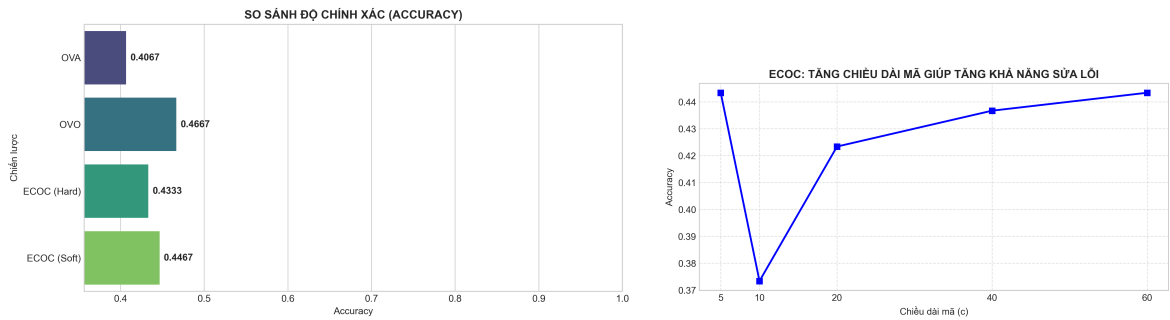
```

12 best_idx = candidates[
13     np.argmax(tie_breaker_scores[sample_idx, candidates])]

```

9.9.3 ECOC và Sức mạnh Sửa lỗi

ECOC mã hoá mỗi lớp thành một *từ mã* trong ma trận $\mathbf{M} \in \{-1, +1\}^{K \times c}$, huấn luyện c bộ phân loại nhị phân con, rồi giải mã bằng khoảng cách Hamming (Hard) hoặc Exponential Loss (Soft, Công thức 8.19 FML). Nhóm đã kiểm chứng hai biến thể và phân tích ảnh hưởng của chiều dài mã c .



(a) So sánh accuracy của OVA, OVO, ECOC Hard và ECOC Soft. (b) Chiều dài mã c tỉ lệ thuận với khả năng sửa lỗi.

Hình 21: Thực nghiệm ECOC trên tập dữ liệu $K = 5$ lớp ($n = 1200$, `class_sep`= 0.8, chia train/test 75/25).

Kết quả thực nghiệm:

- **ECOC Soft** vượt trội ECOC Hard do tận dụng giá trị margin liên tục $f_j(\mathbf{x})$ thay vì nhãn nhị phân rời rạc, giảm mất mát thông tin trong bước giải mã.
- Tăng chiều dài mã c từ 5 lên 60 cải thiện accuracy và bão hoà quanh $c \approx 40$, phù hợp với lý thuyết: khoảng cách Hamming tối thiểu d tăng theo c , cho phép sửa thêm $r_0 = \lfloor (d - 1)/2 \rfloor$ lỗi.

Mã nguồn cài đặt lỗi: Hàm Soft Decoding sử dụng Exponential Loss (Công thức 8.19, FML) và cơ chế sinh ma trận mã hoá tối đa hoá khoảng cách Hamming:

Listing 6: Soft Decoding (Exponential Loss) và sinh ma trận mã hoá trong `ecoc_classifier.py`

```

1 def _exponential_loss_distance(self, code_words, M):
2     losses = np.zeros((code_words.shape[0], M.shape[0]))
3     for l in range(M.shape[0]):
4         mask = (M[l] != 0)
5         margins = M[l, mask] * code_words[:, mask]
6         losses[:, l] = np.sum(np.exp(-margins), axis=1)

```

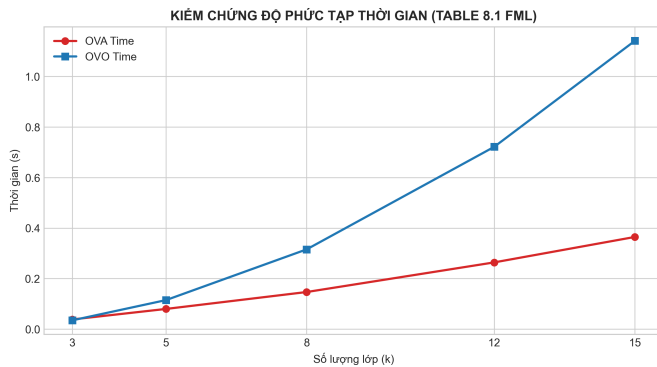
```

7     return losses
8 for attempt in range(100):
9     M = np.random.choice([-1, 1], size=(n_classes, code_size))
10    d = self._minimum_hamming_distance(M)
11    if d > max_min_dist:
12        max_min_dist, best_M = d, M
13    if d >= code_size // 2:
14        break

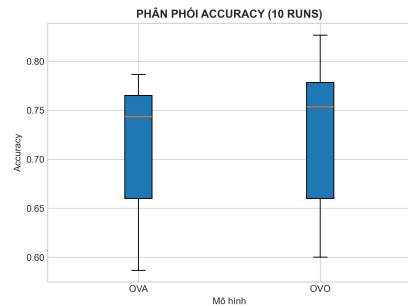
```

9.9.4 So sánh độ phức tạp thời gian và kiểm định thống kê

Để hoàn thiện đánh giá ba chiến lược, nhóm kiểm chứng sự khác biệt độ phức tạp thời gian $\mathcal{O}(K)$ (OVA) so với $\mathcal{O}(K^2)$ (OVO), đồng thời dùng Paired t -test (10 lần chạy độc lập) kiểm định ý nghĩa thống kê của sự chênh lệch accuracy.



(a) Thời gian huấn luyện theo số lớp k .



(b) Phân phối accuracy qua 10 lần chạy.

Hình 22: Độ phức tạp thời gian huấn luyện và kiểm định giả thuyết OVA vs. OVO.

Kết quả thực nghiệm:

- OVO chậm hơn OVA rõ rệt khi k tăng: tại $k = 15$, OVO huấn luyện 105 bộ phân loại nhị phân con trong khi OVA chỉ cần 15, phù hợp với lý thuyết Table 8.1 (FML).
- Paired t -test cho $p > 0.05$, tức **không có sự khác biệt có ý nghĩa thống kê** về accuracy giữa OVA và OVO khi bộ phân loại nền đủ mạnh – phù hợp với nhận định lý thuyết.

9.10 Phân tích kết quả kỳ vọng

Nếu dữ liệu cân bằng và số lớp vừa phải, OVA thường là lựa chọn mạnh vì chỉ cần k mô hình và dự đoán nhanh. Khi bộ học nền có chi phí huấn luyện cao, OVO có thể huấn luyện nhanh hơn do mỗi mô hình chỉ dùng dữ liệu của hai lớp. Tuy nhiên, OVO phải đánh giá $k(k-1)/2$ mô hình khi dự đoán, nên thời gian test có thể lớn hơn.

ECOC có thể vượt OVA/OVO nếu ma trận mã có khoảng cách Hamming đủ lớn và các bộ phân loại bit không sai đồng thời theo cùng một mẫu. Nếu mã quá ngắn hoặc các cột mã tương quan mạnh, khả năng sửa lỗi giảm đáng kể.

Khi dữ liệu mất cân bằng, một mô hình luôn dự đoán lớp đa số vẫn có thể đạt accuracy cao nhưng macro-F1 thấp. Do đó phần nhận xét cuối cùng cần ưu tiên macro-F1, confusion matrix và chất lượng trên các lớp thiểu số thay vì chỉ nhìn accuracy.

10 Kết luận

Đồ án đã hệ thống hóa bài toán phân loại đa lớp từ phát biểu toán học, hàm điểm số, lẽ đa lớp, hàm mất mát lẽ đến các giới hạn tổng quát hóa dựa trên độ phức tạp Rademacher. Phân tích cho thấy số lớp, độ phức tạp của lớp giả thuyết và kích thước lẽ đều ảnh hưởng trực tiếp đến khả năng tổng quát hóa của mô hình.

Về thuật toán, nhóm không kết hợp như multi-class SVM, boosting và cây quyết định học trực tiếp trên không gian đa lớp. Nhóm kết hợp như OVA, OVO và ECOC quy bài toán về nhiều bài toán nhị phân rồi tổng hợp dự đoán. OVA đơn giản và dự đoán nhanh, OVO giảm kích thước từng bài toán con nhưng cần nhiều mô hình hơn, còn ECOC bổ sung tư tưởng mã sửa lỗi để tăng độ bền vững.

Các hướng hiện đại như Extreme Multi-label Classification, cây nhãn, nhúng nhãn, Transformer, CRF và mô hình ngôn ngữ lớn mở rộng phân loại đa lớp sang bối cảnh số nhãn cực lớn hoặc đầu ra có cấu trúc. Đây là các hướng quan trọng trong những ứng dụng thực tế như gán nhãn văn bản, đề xuất sản phẩm, truy hồi thông tin và xử lý ngôn ngữ tự nhiên.

Trong thực nghiệm, cần đánh giá đồng thời độ chính xác, macro-F1, weighted-F1, confusion matrix và thời gian chạy. Không có phương pháp tối ưu tuyệt đối cho mọi dữ liệu; lựa chọn mô hình phụ thuộc vào số lớp, mức mất cân bằng, kích thước dữ liệu, yêu cầu tốc độ và khả năng diễn giải.

Tài liệu

- [1] M. Mohri, A. Rostamizadeh, and A. Talwalkar, *Foundations of Machine Learning*, 2nd ed. MIT Press, 2018.
- [2] T. G. Dietterich and G. Bakiri, “Solving multiclass learning problems via error-correcting output codes,” *Journal of Artificial Intelligence Research*, vol. 2, pp. 263–286, 1995.
- [3] J. Lafferty, A. McCallum, and F. C. N. Pereira, “Conditional random fields: Probabilistic models for segmenting and labeling sequence data,” in *Proceedings of ICML*, 2001.
- [4] B. Taskar, C. Guestrin, and D. Koller, “Max-margin Markov networks,” in *Advances in Neural Information Processing Systems*, 2003.
- [5] I. Tsochantaridis, T. Joachims, T. Hofmann, and Y. Altun, “Large margin methods for structured and interdependent output variables,” *Journal of Machine Learning Research*, vol. 6, pp. 1453–1484, 2005.
- [6] Z. Huang, W. Xu, and K. Yu, “Bidirectional LSTM-CRF models for sequence tagging,” arXiv:1508.01991, 2015.
- [7] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proceedings of NAACL-HLT*, 2019.
- [8] S. Khandagale, H. Xiao, and R. Babbar, “Bonsai: Diverse and shallow trees for extreme multi-label classification,” *Machine Learning*, vol. 109, pp. 2099–2119, 2020.
- [9] R. You, S. Dai, Z. Zhang, H. Mamitsuka, and S. Zhu, “AttentionXML: Label tree-based attention-aware deep model for high-performance extreme multi-label text classification,” in *Advances in Neural Information Processing Systems*, 2019.
- [10] X. Zhang, Q. Lin, Y. Xiong, C. Chen, Y. Chen, and P. N. Bennett, “XR-Transformer: Fast multi-resolution transformer fine-tuning for extreme multi-label text classification,” in *Proceedings of the Web Conference*, 2021.
- [11] Y. Zhu and S. Zamani, “ICXML: An in-context learning framework for zero-shot extreme multi-label classification,” in *Findings of the Association for Computational Linguistics: NAACL*, 2024.
- [12] A. Mao, M. Mohri, and Y. Zhong, “Structured prediction with stronger consistency guarantees,” in *Advances in Neural Information Processing Systems*, 2023.

- [13] Zhijun Chen et al., “Neural-Hidden-CRF: A robust weakly-supervised sequence labeler,” in *Proceedings of ACL*, 2023.
- [14] J. Kim et al., “Exploring nested named entity recognition with large language models: Methods, challenges, and insights,” in *Proc. EMNLP 2024*, Miami, FL, USA, 2024, pp. 8653–8670.
- [15] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.